OPTIMISASI ALGORITMA KOMPRESI LEMPEL-ZIV-WELCH DENGAN TRANSFORMASI BERKAS BURROWS WHEELER

LAPORAN TUGAS AKHIR

Disusun sebagai syarat kelulusan tingkat sarjana

oleh :
Jonathan Marcel T / 13507072



PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2011

LEMBAR PENGESAHAN

OPTIMISASI ALGORITMA KOMPRESI LEMPEL-ZIV-WELCH DENGAN TRANSFORMASI BERKAS BURROWS WHEELER

oleh

Jonathan Marcel T

13507072

Telah disetujui dan disahkan sebagai Laporan Tugas Akhir II

Program Sarjana Teknik Informatika, Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Bandung, 25 Februari 2011

Pembimbing

Dra. Harlili, M.Sc.

NIP. 196311261988031002

ABSTRAK

Saat ini, kebutuhan untuk menjaga kelestarian bahasa-bahasa daerah Nusantara semakin meningkat. Untuk itu, repositori yang menyimpan data seluruh bahasa daerah akan sangat membantu. Selain menyimpan data, keterhubungan antarbahasa daerah juga perlu ditunjukkan. Teknologi web semantik yang menyimpan data berbentuk RDF (*Resource Description Framework*) memungkinkan hal ini. Pada tahun 2009, telah dilakukan penelitian yang mengimplementasikan kamus bahasa daerah dengan *library* ARC, namun masih terbatas pada satu bahasa daerah.

Sebagai pengembangan hasil penelitian tersebut, penelitian Tugas Akhir ini akan meninjau kasus kamus multibahasa. Dengan demikian, data yang harus disimpan akan menjadi sangat banyak, yang mengakibatkan menurunnya performansi sistem. Peningkatan performansi dapat ditinjau dari dua aspek, yaitu penyimpanan dan pencarian (*retrieval*). Pada Tugas Akhir ini, dihasilkan strategi penyimpanan data RDF dalam jumlah besar, spesifik untuk kasus kamus multibahasa.

Berdasarkan hasil analisis, kecenderungan penggunaan kamus bahasa adalah pada satu bahasa saja, namun ada kalanya muncul kebutuhan akan kamus multibahasa. Untuk mengatasi hal ini, digunakan sebuah bahasa antara, yaitu bahasa Indonesia. Selain itu, skema penyimpanan yang dihasilkan ARC tidak cocok untuk kasus kamus multibahasa, dan dari kajian terhadap tiga skema yang digunakan untuk menyimpan RDF, didapatkan bahwa skema *hybrid* merupakan skema yang paling tepat. Untuk meningkatkan performansi lebih jauh lagi, penyimpanan data perlu dilakukan secara terdistribusi untuk mengurangi beban *server*.

Selanjutnya, dilakukan pengujian terhadap hasil analisis dan implementasi. Secara garis besar, pengujian terbagi menjadi dua, yaitu pengujian skema acuan yang dipilih dan pengujian penyimpanan terpusat dan terdistribusi. Dari hasil pengujian, skema *hybrid* terbukti merupakan skema yang memberikan hasil optimal. Penyimpanan secara terdistribusi juga mempercepat *response time* dari *server* terhadap akses yang dilakukan oleh banyak *user*.

Kata kunci: web semantik, penyimpanan RDF, kamus multibahasa

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yesus Kristus, atas bimbingan-Nya dan berkat-Nya sehingga tugas akhir yang berjudul “Optimisasi Algoritma Kompresi Lempel-Ziv-Welch dengan Transformasi Berkas Burrows Wheeler” dapat diselesaikan dengan baik.

Selama masa pengerjaan tugas akhir ini, penulis mendapat banyak dukungan dari berbagai pihak. Oleh karena itu, penulis ingin mengucapkan terimakasih sebesar-besarnya kepada:

1. Papa dan Mama, tanpa dukungan, doa, dan kasih sayang yang begitu besar terhadap penulis, penulis tidak dapat menyelesaikan masa pendidikan di tempat yang jauh dari rumah. Dan juga kepada Koko yang telah banyak mendukung penulis selama bertempat tinggal di Bandung.
2. Ibu Harlili, selaku dosen pembimbing yang senantiasa memberikan dukungan, bimbingan, dan masukan-masukan serta menyemangati penulis agar tepat waktu untuk menyelesaikan tugas akhir.
3. Ratna Kartini Surjono, teman terdekat penulis sejak pertengahan 2010 di Bandung. Terimakasih banyak atas dukungan dan perhatian selama penulis menjalani masa-masa terakhir kuliah di ITB. Semoga hubungan kita terus berjalan dengan baik sampai kapanpun. Maafkan apabila penulis banyak sekali mengecewakan kamu.
4. Teman-teman baik penulis, khususnya teman-teman dari St. Lucia Choir, antara lain Erwin, Alex, Christ, Tandrey, Maxi, Audi, Kevin, Ari, Marco, Ami, Amel, Rina, Janice, Renata, Vina, Tasia, dan teman-teman lainnya. Terimakasih atas masa-masa terindah selama penulis bergabung menjadi St. Lucia Choir. Terimakasih juga karena kalian telah menjadi sahabat yang baik dalam suka maupun duka.
5. Teman-teman yang sering berkumpul di laboratorium Basis Data, khususnya Firdi Mulia dan Aris Feryanto, yang telah banyak sekali membantu dalam mengerjakan tugas akhir ini. Dan juga kepada Hendy, Kevin, James, Adiputra, Tommy, dan lain-lain. Terimakasih atas semua dukungan dan semangat yang telah kalian berikan. Terimakasih telah menjadi teman yang baik, dalam kegiatan akademis maupun non-akademis.

6. Teman-teman himpunan informatika, khususnya teman-teman terdekat di informatika 2007 yaitu Calvin Irwan, Amalfi, Raditya, Dimas, Rachmansyah Budi, Shauma, Edri, Riris, Bhella dan lain-lain. Terima kasih atas kehadiran kalian dalam hidup penulis. Kalian adalah salah satu teman-teman terbaik yang pernah penulis miliki.
7. Segenap staff di kosan Grha Ciumbuleuit, antara lain Pak Idan yang selalu membantu penulis ketika berolahraga, Pak Parman, Mas Ali, dan Ibu Ros yang selalu memberikan semangat dan dukungan kepada penulis, dan juga kepada Bapak-Bapak Satpam seperti Pak Ade, Pak Maman, Pak Yayan, Pak Aang, dan lain-lain yang telah menjaga kosan selama 4 tahun penulis bertempat tinggal disini, dan teman-teman RoomBoy seperti Mas Asep dan kawan-kawan, terimakasih kalian telah menjadi orang yang baik dan jujur terhadap penulis.
8. Ciangsem Aping, Ciangsuk Wei, Ryan, dan Renata. Kalian seperti keluarga kedua penulis selama berada di Bandung. Terimakasih atas dukungan dan bantuannya sehingga penulis dapat berdiri seperti sekarang ini. Semoga keluarga kalian tetap berbahagia sampai selama-lamanya.
9. Teman-teman baik yang sama-sama berasal dari Manado, yaitu Rendy, Yesq, Edwin. Kalian adalah sahabat terbaik saya sejak penulis masih kecil. Juga kepada Crasella, Carol, Irene, Vincent, Raymond, Gerald, Ramon, terimakasih atas dukungannya walaupun kalian berada jauh dari penulis.
10. Seluruh dosen yang pernah mengajari berbagai macam hal kepada penulis selama penulis berkuliah di ITB. Terimakasih atas jasa-jasamu terhadap penulis. Tidak ada yang dapat membalas pengabdian kalian selain ucapan terimakasih yang sebesar-besarnya dari penulis.

Terakhir, penulis juga menyadari bahwa tugas akhir ini masih jauh dari kesempurnaan. Oleh karena itu, saran dan kritik sangat penulis harapkan untuk perbaikan tugas akhir ini di kemudian hari.

DAFTAR ISI

LEMBAR PENGESAHAN.....ii

ABSTRAK.....iii

KATA PENGANTAR.....iv

DAFTAR ISI.....vi

DAFTAR GAMBAR.....ix

DAFTAR TABEL.....xi

BAB I PENDAHULUAN.....I-1

1.1 Latar Belakang.....I-1

1.2 Rumusan Masalah.....I-3

1.3 Tujuan.....I-3

1.4 Batasan Masalah.....I-3

1.5 Metodologi.....I-4

BAB II TINJAUAN PUSTAKA.....II-1

2.1 Teori Data dan Informasi.....II-1

2.2 Teori Pemampatan Data.....II-2

2.3 Metode Pemampatan Data Lempel Ziv Welch (LZW).....II-6

2.4 Transformasi Berkas.....II-8

2.5 Transformasi Burrows Wheeler.....II-9

2.6 Kesimpulan Hasil Tinjauan Pustaka.....II-10

BAB III DESKRIPSI SOLUSI DAN ANALISIS PERANGKAT LUNAK.....III-1

3.1. Kompresi Algoritma Lempel Ziv Welch.....III-1

3.2. Kelemahan Algoritma LZW.....III-3

3.3. Solusi yang Ditawarkan.....III-3

3.4. Analisis Solusi.....III-5

3.5. Faktor-Faktor yang Menentukan.....III-6
3.6. Analisis Perangkat Lunak.....III-7
3.6.1. Deskripsi Perangkat Lunak.....III-7
3.6.2. Spesifikasi Kebutuhan Perangkat Lunak.....III-7
3.6.3. Diagram Use Case.....III-8
3.6.4. Skenario Use Case.....III-8
3.6.5. Realisasi Use Case Tahap Analisis.....III-11
3.6.6. Diagram Kelas Analisis.....III-13
BAB IV PERANCANGAN, IMPLEMENTASI, DAN EVALUASI.....IV-1
4.1 Perancangan.....IV-1
4.1.1. Perancangan Antarmuka.....IV-1
4.1.2. Realisasi Use Case Tahap Perancangan.....IV-2
4.1.3. Diagram Kelas Perancangan.....IV-3
4.2. Implementasi.....IV-4
4.2.1. Lingkungan Implementasi.....IV-4
4.2.2. Batasan Implementasi.....IV-4
4.3 Evaluasi.....IV-4
4.3.1 Evaluasi Perangkat Lunak.....IV-4
4.4 Kesimpulan Hasil Pengujian.....IV-7
BAB V KESIMPULAN DAN SARAN.....V-1
5.1 Kesimpulan.....V-1
5.2 Saran.....V-1
DAFTAR REFERENSI.....x
LAMPIRAN A QUERY UNTUK PENGUJIAN.....1
A.1 Perbandingan Ukuran Repositori.....1

A.1.1 Ukuran Basis Data secara Keseluruhan.....1
A.1.2 Ukuran Tiap Tabel dalam Basis Data.....1
A.2 Pengujian Skema.....2
A.2.1 *Retrieve*.....2
A.2.2 *Insert*.....4
A.2.3 *Update*.....5
A.2.4 Terjemahan.....6
LAMPIRAN B HASIL PENGUJIAN SERVER TUNGGAL DAN TERDISTRIBUSI.....1
B.1 Pengujian Server Tunggal dan Terdistribusi.....1
B.1.1 Hasil Pengujian untuk *Query* 1 Triplet.....1
B.1.2 Hasil Pengujian untuk *Query* Terjemahan Bahasa Jawa-Sunda.....2

DAFTAR GAMBAR

Gambar II-1 *Smart Data Continuum* [DAC03].....II-2
Gambar II-2 Contoh Hirarki Kelas dalam GOLD[MUL09].....II-6
Gambar II-3 Contoh Hirarki Properti dalam GOLD[MUL09].....II-6
Gambar II-4 RDF *Triple* [DAC03].....II-6
Gambar II-5 Contoh HTML Biasa.....II-8
Gambar II-6 Contoh HTML yang Mengandung RDF.....II-8
Gambar II-7 Representasi N3.....II-9
Gambar II-8 Representasi RDF/XML.....II-10
Gambar II-9 Contoh SPARQL.....II-12
Gambar II-10 Representasi *Schema Oblivious*/*Structure Centric* [THE05].....II-13
Gambar II-11 Representasi *Schema Aware*/*Data Centric* [THE05].....II-15
Gambar II-12 Skema *object-relational* dalam PostgreSQL [BRO02].....II-16
Gambar II-13 Representasi *Hybrid* [THE05].....II-17
Gambar II-14 Fragmentasi Vertikal.....II-19
Gambar II-15 Contoh Fragmentasi Vertikal.....II-20
Gambar II-16 Fragmentasi Horizontal.....II-20
Gambar II-17 Contoh Fragmentasi Horizontal.....II-21
Gambar II-18 Fragmentasi *Hybrid*.....II-21
Gambar III-1 Skema Basis Data yang dihasilkan ARC.....III-4
Gambar III-2 Contoh Data Tabel arc_langit_triple.....III-4
Gambar III-3 Contoh Data Tabel arc_langit_s2val.....III-4
Gambar III-4 Contoh Data Tabel arc_langit_id2val.....III-5
Gambar III-5 Contoh Data Tabel arc_langit_o2val.....III-5
Gambar III-6 Contoh Data Tabel arc_langit_g2t.....III-5
Gambar III-7 Redundansi pada Skema ARC.....III-6
Gambar III-8 *Entity Relationship Diagram* (ERD).....III-13
Gambar III-9 Skema Basis Data LangIT.....III-13
Gambar III-10 Ilustrasi Penyimpanan Terdistribusi.....III-19
Gambar IV-1 Skema Uji - *Schema Aware*.....IV-2
Gambar IV-2 Skema Uji - *Schema Oblivious*.....IV-2

Gambar IV-3 Perbandingan Ukuran Repositori.....IV-3
Gambar IV-4 Rincian Ukuran Repositori - *Schema Oblivious*.....IV-3
Gambar IV-5 Rincian Ukuran Repositori - *Schema Aware*.....IV-4
Gambar IV-6 Rincian Ukuran Repositori - *Hybrid*.....IV-4
Gambar IV-7 Perbandingan *Query Response Time - Retrieve*.....IV-6
Gambar IV-8 Perbandingan *Query Response Time – Insert*.....IV-8
Gambar IV-9 Perbandingan *Query Response Time – Update*.....IV-10
Gambar IV-10 Perbandingan *Query Response Time* – Terjemahan.....IV-11
Gambar IV-11 Grafik Perbandingan Waktu Respons Rata-rata - *Query* 1 Triplet IV-13
Gambar IV-12 Grafik Perbandingan Waktu Respons Rata-rata - *Query* Terjemahan IV-13

DAFTAR TABEL

Tabel II-1 Triplet RDF dari Gambar II-8.....II-9
Tabel II-2 Contoh Triplet RDF.....II-9
Tabel III-1 Tabel [Bahasa_Daerah]_Triple_Resource.....III-14
Tabel III-2 Tabel [Bahasa_Daerah]_Triple_Literal.....III-14
Tabel III-3 Tabel [Bahasa_Daerah]_Direct_Translation.....III-15
Tabel III-4 Tabel [Bahasa_Daerah]_ID_To_Val.....III-15
Tabel III-5 Tabel Indonesia_ID_To_Val.....III-16
Tabel III-6 Tabel Indonesia_Triple_Resource.....III-16
Tabel III-7 Tabel Indonesia_Triple_Literal.....III-16
Tabel III-8 Tabel Languages.....III-17
Tabel III-9 Tabel URI_Prefixes.....III-17
Tabel IV-1 Perbandingan *Query Response Time - Retrieve*.....IV-5
Tabel IV-2 Perbandingan *Query Response Time – Insert*.....IV-7
Tabel IV-3 Perbandingan *Query Response Time – Update*.....IV-9
Tabel IV-4 Perbandingan *Query Response Time* – Terjemahan.....IV-11

BAB I
PENDAHULUAN

Bab ini akan berisi beberapa penjelasan pendahuluan mengenai topik yang diajukan pada tugas akhir ini, antara lain terdiri atas latar belakang penelitian, rumusan masalah, tujuan penelitian, batasan masalah, metodologi penelitian serta sistematika pembahasan tugas akhir.

1.1 Latar Belakang

Sejak dahulu kala, informasi merupakan salah satu kebutuhan manusia yang paling mendasar. Salah satu penyebab mengapa informasi ini sangatlah penting karena manusia memiliki kodrat sebagai makhluk sosial. Pada dasarnya manusia memerlukan informasi untuk menentukan tindakan apa yang harus dilakukan. Informasi juga merupakan hal yang vital di dalam proses komunikasi.

Data merupakan informasi yang belum diolah, artinya data ini hanya merupakan kumpulan fakta yang dapat berupa angka, karakter, simbol, gambar, dan lain sebagainya. Data ini merupakan deskripsi dari sesuatu kejadian yang dihadapi. Data yang telah diproses menjadi sesuatu yang memiliki makna maka akan disebut sebagai informasi[HAN03]. Kebanyakan sumber informasi menyediakan informasi masih dalam bentuk data, dan kemudian akan diolah di otak manusia yang menerimanya sehingga menjadi informasi yang bermanfaat.

Data maupun informasi akan disimpan atau diarsip sehingga dapat digunakan kembali di kemudian hari. Penyimpanan data atau informasi dapat menggunakan media penyimpanan, misalnya *notes*, *harddisk*, *flashdisk*, CD/DVD, maupun dengan menggunakan media penyimpanan lainnya.

Seiring dengan perkembangan teknologi, pertumbuhan jumlah data maupun informasi berkembang lebih pesat dibandingkan dengan kapasitas media penyimpanan. Karena keterbatasan kapasitas penyimpanan tersebut, maka muncul gagasan untuk menyimpan data atau informasi tersebut dengan lebih efisien[WIT99]. Dengan menyimpan data secara lebih efisien, maka manusia dapat menghemat ruang penyimpanan sehingga jumlah data yang disimpan bisa lebih banyak. Salah satu solusi untuk menyimpan data secara lebih efisien ialah dengan menggunakan teknik kompresi data.

Ide dasar dari pemampatan data ialah banyaknya redundansi (pengulangan) pada representasi data. Pada dasarnya, kompresi data atau pemampatan data ialah memampatkan ukuran data sehingga menjadi lebih kecil dibandingkan sebelumnya. Hal ini bisa dilakukan karena ukuran data akan di-*encode* menjadi bit-bit yang lebih kecil dengan menggunakan metode tertentu.

Ada banyak algoritma pemampatan data, diantaranya yang terkenal antara lain: Huffman Coding, LZ77 (*Sliding Window)*, LZ78, dan LZW. Pada tugas akhir ini digunakan algoritma LZW karena algoritma ini memiliki beberapa kelebihan antara lain merupakan perbaikan dari algoritma LZ77 dan LZ78, mampu mengkompresi data dengan cepat, serta tidak rumit untuk diimplementasikan. Namun, algoritma ini memiliki kelemahan salah satunya ialah sulitnya memanajemen tabel string. Masalah inilah yang menyebabkan algoritma LZW harus mengalami proses optimasi agar memberikan hasil yang lebih baik.

Untuk mengoptimasi algoritma pemampatan data, salah satu metode yang digunakan adalah metode transformasi Burrows Wheeler[SIM08]. Optimasi algoritma pemampatan data bertujuan untuk menghasilkan data yang dimampatkan secara lebih cepat maupun dengan ukuran yang lebih kecil dibandingkan dengan kinerja algoritma sebelum dioptimasi. Metode transformasi Burrows Wheeler mentransformasikan isi dari file teks berdasarkan permutasi seluruh kemungkinan rotasi dari karakter-karakter di dalam file teks. Hasilnya ialah file teks yang akan dimampatkan mengalami perubahan bentuk, dimana bagian-bagian yang memiliki kemiripan akan ditempatkan secara berdekatan. Pada tugas akhir ini digunakan optimasi dengan metode transformasi Burrows Wheeler karena metode ini tidak membutuhkan kamus tambahan sehingga tidak dibutuhkan waktu yang lama dalam proses implementasinya.

Tujuan dari tugas akhir ini yaitu membandingkan bagaimana kinerja algoritma LZW pra maupun pasca proses optimasi. Algoritma tersebut akan mengalami optimasi dengan metode transformasi Burrows Wheeler, sehingga hasil pemampatan datanya secara teoritis akan menjadi lebih baik. Untuk proses perbandingan hasil optimasinya, algoritma LZW akan diimplementasi ke dalam suatu perangkat lunak sederhana, sehingga memudahkan proses pengujian dan evaluasi terhadap kinerja algoritma sebelum maupun sesudah dioptimasi.

1.2 Rumusan Masalah

Ide yang mendasari pengerjaan tugas akhir ini ialah untuk melakukan optimasi terhadap algoritma kompresi data yang cukup popular, yaitu algoritma Lempel Ziv Welch (LZW). Optimasi algoritma diharapkan dapat menghasilkan data hasil kompresi yang lebih termampatkan.

Secara rinci, tahapan dari rumusan masalah di atas ialah:

1. Bagaimana melakukan optimasi terhadap kinerja algoritma LZW dengan menggunakan metode transformasi Burrows Wheeler.
2. Bagaimana melakukan pengukuran tingkat kinerja algoritma LZW pada saat pemampatan data baik sebelum maupun setelah mengalami optimisasi.
3. Bagaimana kinerja algoritma tersebut sebelum maupun setelah mengalami optimisasi.

1.3 Tujuan

Tujuan yang ingin dicapai dari pengerjaan tugas akhir ini ialah membandingkan kinerja algoritma LZW sebelum dan sesudah dioptimasi sehingga dapat diketahui manakah hasil yang lebih baik sebelum atau sesudah dioptimasi.

Adapun tahapan untuk mencapai tujuan tersebut ialah:

1. Mempelajari dan memahami algoritma LZW dan transformasi Burrows Wheeler.
2. Melakukan optimasi terhadap kinerja algoritma LZW dengan menggunakan transformasi Burrows Wheeler.
3. Melakukan implementasi algoritma LZW dan transformasi Burrows Wheeler untuk melakukan pengujian kinerja algoritma.
4. Membandingkan hasil pengujian kinerja algoritma sebelum maupun sesudah dioptimasi dengan transformasi Burrows Wheeler.

1.4 Batasan Masalah

Batasan masalah dalam studi dan penyusunan tugas akhir ini adalah:

a. Implementasi yang akan dilakukan terbatas pada pembuatan kamus kosakata.

b. Implementasi akan dilakukan pada DBMS, bukan dengan membuat *file system* dari nol.

1.5 Metodologi

Dalam mengerjakan tugas akhir ini, ada beberapa langkah yang dilakukan, yaitu:

1. Studi Literatur

Pada tahap ini dilakukan pembelajaran mengenai teori dasar yang digunakan pada tugas akhir ini, antara lain tentang teori kompresi data, algoritma LZW, dan transformasi Burrows Wheeler. Literatur yang digunakan antara lain bahan dari situs internet, makalah ilmiah, buku teks, maupun tugas akhir yang relevan dengan topik kompresi data.

2. Analisis

Pada tahap ini dilakukan analisis mengenai optimasi yang dilakukan, antara lain hipotesis optimasi dengan menggunakan transformasi Burrows Wheeler dapat meningkatkan kinerja algoritma kompresi LZW. Pada tahap ini juga dilakukan analisis terhadap perangkat lunak yang dibangun untuk melakukan pengujian terhadap kinerja kedua algoritma sebelum dan sesudah dioptimasi.

3. Perancangan dan Implementasi

Pada tahap ini dilakukan perancangan serta implementasi algoritma kompresi dalam bentuk perangkat lunak. Perangkat lunak yang dibangun berdasarkan hasil analisis yang telah dilakukan sebelumnya. Perangkat lunak ini berguna untuk membantu mengukur kinerja algoritma sebelum maupun sesudah dioptimasi dengan transformasi Burrows Wheeler.

4. Pengujian dan Evaluasi

Pada tahap ini dilakukan pengujian perangkat lunak yang telah dibuat untuk menguji apakah perangkat lunak bisa berfungsi sesuai rancangan yang telah dibuat. Setelah itu akan dilakukan evaluasi kinerja algoritma LZW sebelum maupun sesudah dioptimasi.

5. Kesimpulan dan Saran

Pada tahap ini dilakukan penarikan kesimpulan studi yang telah dilakukan. Akan ditarik kesimpulan apakah hipotesis awal penelitian terbukti ataupun tidak terbukti. Kemudian akan diberikan saran bagi penelitian selanjutnya yang memiliki relevansi dengan tugas akhir ini.

1.6 Sistematika Pembahasan

BAB II
TINJAUAN PUSTAKA

Bab ini akan berisi pembahasan mengenai teori-teori hasil studi literatur yang melandasi pembuatan tugas akhir ini. Hal-hal yang akan dibahas diantaranya ialah teori data dan informasi, teori pemampatan (kompresi) data, algoritma Lempel-Ziv-Welch, transformasi berkas, transformasi Burrows wheeler, serta kesimpulan hasil tinjauan pustaka yang telah dilakukan.

2.1 Teori Data dan Informasi

Pada dasarnya data dapat diartikan sebagai representasi dari fakta. Awalnya, data hanyalah merupakan kumpulan fakta yang belum memiliki makna. Untuk memperoleh makna dari suatu data, data tersebut harus mengalami pemrosesan, sehingga akan berubah menjadi informasi[WIT99].

Teori informasi yang dikembangkan oleh Claude E. Shannon mempelajari aspek kuantitatif dari informasi[HAN03]. Dengan menggunakan teori ini, dapat dihitung banyaknya informasi yang terdapat dalam suatu data dengan angka yang tepat. Karena itulah didefinisikan unit dari informasi yang disebut dengan bit. Saat ini sudah menjadi hal yang lumrah untuk menyebutkan ukuran dari suatu informasi dalam satuan bit. Teori informasi banyak diaplikasikan di berbagai bidang, diantaranya pemrosesan bahasa alami, kriptografi, komputasi kuantum, inferensi statistik, dan lain-lain.

Dalam ilmu komputer, data digunakan sebagai informasi yang dibutuhkan oleh komputer agar dapat menjalankan fungsinya. Di dalam komputer, data disimpan dalam format *binary* dan direpresentasikan oleh kumpulan angka 0 dan 1. Setiap angka 0 maupun 1 menghabiskan ruang sebesar 1 bit di dalam media penyimpanan.
Jumlah data yang terus bertambah tidak diimbangi dengan peningkatan kapasitas media penyimpanan data. Oleh karena itu, manusia mencari cara agar dapat menyimpan data secara lebih efektif untuk memanfaatkan kapasitas media penyimpanan secara optimal. Salah satu gagasan yang muncul ialah bagaimana

merepresentasikan data dalam bentuk yang lebih ringkas, sehingga data dapat disimpan dalam ukuran yang lebih kecil[HAN03].

2.2 Teori Pemampatan Data

Pemampatan data dapat diartikan sebagai proses mengubah aliran data masukan ke dalam aliran data lain yang lebih kecil. Pemampatan data, atau disebut juga kompresi data memiliki berbagai matode berdasarkan ide yang berbeda-beda. Proses pemampatan data menjadi populer dalam perkembangan ilmu komputer dikarenakan oleh hal-hal berikut[SAL07]:

1. Manusia memiliki kecenderungan untuk mengumpulkan data secara terus-menerus tanpa menghapus data sebelumnya. Hal ini tentunya menjadi masalah karena kapasitas media penyimpanan semakin lama semakin berkurang. Akibatnya, data baru yang akan disimpan tidak lagi memiliki tempat penyimpanan. Dengan menggunakan metode pemampatan data, jumlah data yang disimpan dalam media penyimpanan dapat ditingkatkan.
2. Waktu transfer data dapat dipercepat apabila data berukuran lebih kecil. Hal ini sangat bermanfaat, terutama karena pada era internet sekarang, kecepatan transfer data menjadi *constraint* yang sangat penting untuk para pengguna internet. Dengan menggunakan metode pemampatan data yang sesuai, data yang akan ditransfer dapat dikurangi ukurannya dan menyebabkan waktu transfer data menjadi berkurang.

Prinsip dasar metode pemampatan data ialah dengan menyingkirkan bagian yang dianggap redundan dari aliran data masukan. Hal ini berdasarkan prinsip bahwa data memiliki struktur tertentu, dimana struktur ini dapat dimanipulasi untuk mendapatkan representasi data dalam ukuran yang lebih kecil. Karena struktur data dapat berubah antara berbagai jenis data, maka tidak ada algoritma pemampatan data yang bekerja secara optimal untuk seluruh jenis data.

Metode pemampatan data dapat dibagi menjadi dua bagian, yaitu pemampatan data lossless dan pemampatan data lossy[SAL07]. Pada pemampatan data lossy, data yang dimampatkan akan memiliki perbedaan dengan data asal karena pada metode ini sebagian informasi pada data asal akan dihilangkan. Metode pemampatan data lossy

banyak digunakan untuk memampatkan data video, suara, dan gambar. Hal ini dikarenakan pada tipe data tersebut, tidak akan terlihat perbedaan yang signifikan apabila sebagian informasi dari tipe data tersebut dihilangkan. Namun lain halnya pada data teks, dimana perubahan informasi, bahkan hanya sebesar 1 bit, dapat mengakibatkan perbedaan yang signifikan antara data awal dan data hasil pemampatan. Karena itulah, pada pemampatan data teks sebaiknya menggunakan metode lossless, yaitu data asal sama dengan data hasil pemampatan.

Beberapa hal yang dapat digunakan untuk mengukur kinerja dari sebuah algoritma pemampatan data ialah sebagai berikut[SAL07]:

1. Rasio pemampatan data (*compression ratio*)

Adalah ukuran hasil pemampatan data dibagi dengan ukuran file semula. Semakin kecil nilai rasio pemampatan data, semakin efektif kinerja algoritma kompresi tersebut. Sebagai contoh, apabila nilai rasio 0,7 berarti algoritma menghasilkan data keluaran berukuran 70% dari ukuran semula.

$$R_o = \frac{Ukuran\ \ keluaran\ \ aliran\ \ data}{Ukuran\ \ masukan\ \ aliran\ \ data}$$

2. Faktor pemampatan data (*compression factor*)

Adalah inverse dari rasio pemampatan data. Apabila nilainya lebih besar dari satu, maka algoritma pemampatan data tersebut tidak layak untuk digunakan.. Karena apabila nilai lebih dari satu maka terjadi ekspansi data, dan nilai kurang dari satu maka terjadi pemampatan data.

$$F_o = \frac{Ukuran\ \ masukan\ \ aliran\ \ data}{Ukuran\ \ keluaran\ \ aliran\ \ data}$$

3. Kecepatan kompresi (*compression speed*)

Adalah perbandingan ukuran berkas file yang akan dimampatkan dengan waktu yang dibutuhkan untuk memampatkan berkas file tersebut. Apabila kecepatan kompresi kecil maka waktu yang dibutuhkan untuk memampatkan data menjadi lama sehingga algoritma pemampatan data tersebut tidak banyak digunakan.

$$\text{Kecepatan kompresi} = \frac{Ukuran\ \ file\ \ semula}{Waktu\ \ kompresi}$$

4. Kecepatan dekompresi (*decompression speed*)

Adalah perbandingan ukuran berkas file hasil dimampatkan dengan waktu yang dibutuhkan untuk mendekompresi berkas file tersebut menjadi bentuk file awal. Algoritma pemampatan data yang baik haruslah memiliki kecepatan kompresi dan dekompresi yang saling berimbang.

$$\text{Kecepatan dekompresi} = \frac{Ukuran\ \ file\ \ hasil\ \ pemampatan}{Waktu\ \ dekompresi}$$

5. Karakter per bit (*Bit per character*/BPC) [SHA48]

Menghitung BPC erat kaitannya dengan teori entropi informasi dari Shannon. BPC merepresentasikan jumlah minimum bit yang diperlukan untuk meng-*encode* file. Persamaan yang digunakan untuk menghitung BPC ialah:

$$H(X) = -\sum_{i=0}^{N-1} p_i \log_2 p_i$$

Dimana i menyatakan karakter ke-i dan N menyatakan jumlah jenis karakter. Makin kecil nilai H(X) maka nilai entropi semakin kecil, artinya algoritma kompresi semakin baik.

Setelah itu pemampatan data mulai memanfaatkan redundansi pada representasi karakter ASCII. Representasi karakter dengan ASCII memiliki redundansi yang besar, karena ASCII mengkodekan setiap karakter dengan panjang yang tetap. Dengan memanfaatkan redundansi tersebut, muncul gagasan untuk melakukan pengkodean karakter dengan panjang bit variatif, yang dikenal dengan sebutan *varibale size code*.
Untuk mendapatkan kinerja yang baik dari pengkodean bit dengan panjang variatif, maka diperlukan pengetahuan tentang jumlah kemunculan setiap karakter dalam suatu berkas file, agar setiap karakter dapat direpresentasikan dengan panjang kode yang berbeda. Karakter yang memiliki tingkat kemunculan tinggi direpresentasikan dengan kode yang lebih pendek, sedangkan karakter yang memiliki tingkat kemunculan rendah direpresentasikan dengan kode yang lebih panjang. Itulah mengapa pendekatan statistik digunakan, yaitu untuk menghitung kemunculan setiap karakter pada file. Salah satu pendekatan statistik yang populer ialah metode pemampatan data Huffman.

Karena pendekatan secara statistik sangat bergantung kepada permodelan statistik yang dibuat, maka manusia menciptakan pendekatan lain, yaitu pendekatan dengan

menggunakan kamus. Pendekatan dengan menggunakan kamus tidak bergantung pada model statistik dan tidak menggunakan kode dengan panjang variatif (*variable size code*). Pada pendekatan ini yang dilakukan ialah membangun sebuah kamus berdasarkan berkas masukan, dan kamus ini berguna untuk melakukan encoding string masukan menjadi token. Kamus ini dapat bersifat statis, yaitu kamus mengizinkan penambahan simbol saja, maupun bersifat adaptif, yaitu kamus mengizinkan penambahan simbol maupun pengurangan simbol dari dalam kamus.
Langkah-langkah umum pada pemampatan data dengan menggunakan pendekatan kamus ialah:

1. Melakukan pembacaan teks
2. Membuat kamus berdasarkan kemunculan kata pada berkas
3. Mengkodekan setiap kata berdasarkan indeks lokasi kata pada kamus

Sebelum mengambil tugas akhir dengan topik ini, telah ditelaah beberapa literatur yang memiliki keterkaitan dengan penelitian di bidang algoritma pemampatan data. Beberapa literatur yang terkait adalah:

1. Tugas akhir Arinta Praimandini Auza, berjudul “Analisis Perbandingan Algoritma LZ78 dan *Arithmetic Coding* pada Kompresi File Teks”

Pada tugas akhir ini saudari Arinta membandingkan kinerja algoritma kompresi LZ78 dan algoritma kompresi *Arithmetic Coding*. Hasil yang diperoleh ialah algoritma *Arithmetic Coding* mengkompresi file menjadi lebih kecil, tetapi algoritma LZ78 lebih unggul dari segi waktu kompresi. Selain itu saudari Arinta menyimpulkan bahwa algoritma LZ78 tidak cocok untuk mengkompresi file berukuran besar, karena semakin besar ukuran file algoritma LZ78 semakin tidak akurat.

2. Tugas akhir Adi Purwanto Sujarwadi, berjudul “Optimasi Kinerja Algoritma Pemampatan Data Lempel-Ziv dengan Metode Transformasi Burrows Wheeler”

Pada tugas akhir ini, saudara Adi melakukan optimasi pada algoritma LZ77 (*sliding window)* dengan menggunakan metode transformasi Burrows Wheeler. Hasil yang diperoleh ialah, preproses teks dengan algoritma Burrows Wheeler dapat meningkatkan kinerja algoritma kompresi LZ77, yaitu menghasilkan file hasil

kompresi yang berukuran lebih kecil. Selain itu saudara Adi menyimpulkan bahwa transformasi Burrows Wheeler justru dapat mengubah bentuk file menjadi file yang sulit dikompresi, contohnya ialah file dalam kategori *calgalry corpus.*

Oleh karena itu, tugas akhir yang diambil secara ringkas melanjutkan tugas akhir saudara Adi. Karena tugas akhir saudara Adi menggunakan algoritma LZ77, maka tugas akhir ini menggunakan algoritma LZW yang merupakan penyempurnaan dari algoritma LZ77. Selain itu dilakukan optimasi pada dua jenis algoritma pemampatan data, yaitu algoritma Huffman, yang mewakili metode statistik, dan algoritma LZW, yang mewakili metode kamus. Kedua-duanya dioptimasi dengan algoritma transformasi Burrows Wheeler.

2.3 Metode Pemampatan Data Lempel Ziv Welch (LZW)

Algoritma pemampatan data Lempel Ziv Welch adalah algoritma pemampatan data dengan menggunakan pendekatan kamus[ZIV77]. Kamus ini digunakan untuk menyimpan string yang telah ditemukan sebelumnya. Algoritma yang ditemukan oleh Abraham Lempel, Jacob Ziv, dan Terry Welch ini dipublikasikan sebagai implementasi yang lebih baik dari algoritma LZ78.

Untuk memahami algoritma LZW secara lebih mendalam, pertama-tama akan dibahas perbandingan algoritma LZW dengan algoritma LZ78. Pada awalnya kamus pada algoritma LZ78 kosong dan ukurannya terbatas oleh memori yang dapat digunakan. Pemampatan data dengan menggunakan algoritma LZ78 ialah sebagai berikut[SAL07]:

1. Kamus diawali dengan karakter null yang memiliki indeks 0
2. Baca karakter berikutnya pada file input. Setiap string yang ditambahkan pada kamus akan diberi indeks 1, 2, dan seterusnya.
3. Saat karakter x dibaca dari file input, kamus mencari sebuah token yang berisi string x. Token bisa ditulis dalam bentuk (X,Y), dimana X berupa bilangan bulat yang menunjukkan penunjuk pada kamus, dan Y berupa string.
    a) Jika string tidak ditemukan, maka token (0,x) ditambahkan di dalam kamus.

b) Jika string x ditemukan, misalnya pada posisi k, maka string berikutnya y dibaca, dan kamus mencari token yang berisi xy. Jika tidak ditemukan, maka token (k,y) ditambahkan di dalam kamus. Sedangkan apabila karakter ditemukan, maka proses pembacaan berlanjut terus hingga ditemukan karakter yang tidak berada di dalam kamus.

4. Kemudian ulangi dari langkah kedua hingga seluruh isi file terbaca.

Sedangkan pada algoritma pemampatan data LZW, kamus tidak diinisialisasi dengan karakter null, melainkan telah diisi dengan 0-255 karakter ASCII atau 256 8-bit byte. Perbedaan yang lain ialah, token pada LZW hanya terdiri atas satu elemen, yaitu penunjuk pada kamus, karena kamus pada LZW sudah diinisialisasi sehingga karakter yang akan dibaca berikutnya pasti telah ada di dalam kamus. Prinsip utama algoritma ini ialah memampatkan karakter satu per satu dan mengakumulasikannya ke dalam string I Pemampatan data dengan algoritma LZW adalah sebagai berikut[SAL07]:

1. Inisialisasi kamus dengan 256 entri di dalam kamus
2. Baca karakter berikutnya pada file input.
3. Setiap karakter dikonkatenasi dengan string I.

a) Apabila I terus menerus ditemukan di dalam kamus, maka proses pembacaan terus berlanjut.
b) Apabila konkatenasi karakter dengan string I tidak ditemukan di dalam kamus, misalnya Ix, maka kamus menambahkan token baru di dalam kamus, yaitu penunjuk ke string I, menyimpan string Ix di dalam kamus, dan menginisialisasi string I ke karakter x.

4. Ulangi mulai langkah yang kedua hingga pembacaan file input selesai.

LZW dipilih sebagai algoritma kompresi dalam tugas akhir ini karena beberapa alasan, antara lain:

1. LZW sangat cocok untuk digunakan pada kompresi berkas file teks, karena sifatnya yang *lossless*.
2. LZW memiliki waktu kompresi yang cepat dan mudah untuk diimplementasikan.

3. Kamus pada LZW sudah diinisialisasi sehingga menjadikan proses kompresi lebih cepat.

Namun algoritma LZW memiliki kelemahan diantaranya:

1. Pada berkas file yang karakter-karakternya tersebar acak, algoritma LZW justru dapat menyebabkan ukuran file menjadi lebih besar.
2. Manajemen tabel string, yaitu kamus akan menjadi sulit apabila karakter-karakter di dalam file tersebar secara acak.

Kelemahan-kelemahan itulah yang menyebabkan algoritma LZW akan dioptimasi dengan transformasi Burrows Wheeler.

2.4 Transformasi Berkas

Transformasi berkas adalah proses melakukan modifikasi pada berkas sehingga mengubah berkas menjadi bentuk yang lebih mudah untuk dimampatkan. Transformasi berkas biasanya dilakukan sebelum memampatkan data agar bisa mendapatkan hasil pemampatan data yang lebih baik.

Ada 3 jenis transformasi yang seringkali digunakan dalam memproses berkas sebelum dimampatkan, yaitu[RAD09]:

1. Transformasi Burrows Wheeler

Transformasi Burrows Wheeler menggunakan matriks untuk menyusun permutasi kemunculan karakter dalam berkas, sehingga akhirnya pola repetisi yang tadinya terletak secara tersebar dalam berkas teks dapat dikumpulkan pada lokasi yang berdekatan.

2. Transformasi Star

Transformasi Star berdasarkan pada pendefinisian tanda yang unik unuk setiap karakter dan menggantikan karakter tersebut dengan karakter khusus, contohnya karakter *. Transformasi ini menggunakan kamus untuk melakukan transformasi pada berkas.

3. Transformasi LIPT (Length Index Preserving Transform)

Transformasi LIPT merupakan pengembangan dari transformasi Star. Transformasi LIPT menggunakan kamus dan memiliki kecepatan yang lebih baik dibandingkan transformasi Star. Namun sayangnya transformasi ini belum popular digunakan.

2.5 Transformasi Burrows Wheeler

Transformasi Burrows Wheeler adalah salah satu metode yang digunakan untuk melakukan transformasi pada berkas teks. Transformasi yang terjadi ialah menjadikan bagian-bagian yang banyak berulang pada teks menjadi berdekatan. Pada proses transformasi, berkas teks hasil transformasi akan menjadi lebih besar karena ditambahkan variabel lokasi baris pada hasil transformasinya. Bagian yang saling berulang akan diletakkan berdekatan sehingga menjadikan berkas teks tersebut lebih mudah untuk dimampatkan[SIM08].

Cara kerja algoritma transformas Burrows Wheeler adalah sebagai berikut:

1. Membaca string x
2. Membuat tabel yang berisi semua kemungkinan rotasi dari string x
3. Mengurutkan kolom tabel secara alfabetis
4. Mengembalikan kolom terakhir dan nomor baris yang mengandung berkas awal sebagai berkas yang telah ditransformasi

Cara untuk mengembalikan berkas hasil transformasi adalah sebagai berikut:

1. Membuat sebuah tabel kosong
2. Membaca string yang ditransformasi y
3. Ulangi sebanyak panjang y:

a) Masukkan string y sebagai kolom sebelum kolom pertama
b) Lakukan pengurutan secara alfabetis

4. Jadikan baris yang bersesuaian dengan nomor baris yang terdapat pada berkas yang telah ditransformasi sebagai keluaran

Transformasi Burrows Wheeler memiliki beberapa keunggulan sehingga digunakan pada tugas akhir ini antara lain:

1. Transformasi Burrows Wheeler mampu menjadikan berkas file yang memiliki karakter-karakter berulang dengan posisi yang berdekatan. Hal ini dapat menjadi jawaban atas kelemahan pada algoritma LZW.
2. Transformasi Burrows Wheeler sederhana untuk diimplementasikan karena tidak membutuhkan kamus tambahan pada proses transformasi. Pada

transformasi Star maupun LIPT perlu dibangun sebuah kamus yang berisi kata-kata yang akan ditransformasi. Hal ini tentu akan menjadi kendala karena jumlah kata di dalam kamus dapat mencapai 400.000 buah kata.

3. Karena tidak menggunakan kamus tambahan, maka proses transformasi Burrows Wheeler tidak memiliki keterbatasan terhadap bahasa yang digunakan pada file, karena hanya melakukan permutasi terhadap isi file tersebut tanpa mencocokkan kata-kata berdasarkan kamus yang dibangun sebelumnya.

2.6 Kesimpulan Hasil Tinjauan Pustaka

Setelah melakukan tinjauan pustaka seperti yang diuraikan pada subbab 2.1 hingga 2.4, maka kesimpulan sementara yang dapat diambil terkait tugas akhir ini adalah sebagai berikut:

Teknologi web semantik memungkinkan mesin untuk memahami makna dari data. Hal ini dilakukan dengan menambahkan informasi semantik pada data, sehingga data yang tersimpan menjadi data yang cerdas (*smart data*). Salah satu bentuk *smart data* adalah ontologi. Ontologi menyediakan sejumlah aturan untuk merepresentasikan konsep-konsep serta keterhubungan antarkonsep dalam suatu domain. Dengan demikian, teknologi web semantik membantu memecahkan salah satu permasalahan yang dihadapi arsitektur teknologi informasi saat ini, yaitu *poor content aggregation* atau kurangnya informasi yang menunjukkan keterkaitan antardata yang tersimpan di berbagai sumber yang berlainan.

Salah satu contoh nyata yang diangkat sebagai topik tugas akhir ini adalah ontologi bahasa. Hingga saat ini, banyak ahli linguistik yang telah mendefinisikan terminologi-terminologi bahasa, namun informasi mengenai hal ini tidak dikomunikasikan untuk dapat dipakai bersama. Di sinilah peran web semantik, yakni sebagai sarana untuk melakukan *information sharing* dan *knowledge discovery* bagi para ahli linguistik. Salah satu cara yang dapat dilakukan adalah dengan memetakan terminologi-terminologi yang sudah ada ke suatu ontologi bahasa yang bersifat lebih umum, yaitu GOLD. Melalui GOLD, sebuah terminologi dalam suatu ontologi bahasa dapat dicari padanannya dalam ontologi bahasa lain.

Terlepas dari berbagai kemudahan yang disediakan oleh web semantik, terdapat satu permasalahan yang saat ini banyak diteliti untuk dicari solusinya, yakni penyimpanan data. RDF merupakan sebuah standar untuk merepresentasikan *smart data*. Untuk kasus ontologi bahasa, data RDF akan sangat besar, seperti gambaran pada subbab Latar Belakang. Oleh karena itu, perlu dicari suatu cara untuk mengelola data RDF yang berjumlah sangat banyak, dengan tetap menjaga performansi sistem (kemudahan dan kecepatan akses data). Sejauh ini, berdasarkan hasil kajian terhadap penelitian terkait untuk penyimpanan data RDF, secara umum representasi *hybrid* memberikan performansi yang lebih baik daripada representasi lain. Namun, kajian lebih lanjut akan dilakukan terhadap kelebihan dan kekurangan masing-masing representasi, terutama terkait dengan kriteria dari ontologi bahasa.

Basis data terdistribusi merupakan salah satu cara yang dapat digunakan untuk meningkatkan performansi sistem yang menangani data dalam jumlah yang sangat besar, yaitu dengan menyimpan data pada lebih dari satu *site*. Basis data terdistribusi dapat diklasifikasikan menjadi dua jenis, yaitu basis data homogen dan heterogen, bergantung pada skema basis data dan DBMS yang digunakan oleh masing-masing *site*. Terdapat berbagai alternatif penyimpanan data terdistribusi, yaitu fragmentasi, replikasi, maupun gabungan dari keduanya. Pemilihan cara penyimpanan didasarkan pada kebutuhan dan karakteristik sistem yang akan dibangun.

BAB III
DESKRIPSI SOLUSI DAN ANALISIS PERANGKAT LUNAK

Bab ini akan membahas analisis dari masalah yaitu bagaimana kinerja kompresi LZW dapat ditingkatkan melalui transformasi Burrows Wheeler. Selain diberikan analisis terhadap solusi yang diusulkan, bab ini juga akan membahas tentang analisis perangkat lunak yang akan dibuat, antara lain mengenai diagram kelas analisis, dan perancangan use case tahap analisis.

3.1. Kompresi Algoritma Lempel Ziv Welch

Algoritma Lempel Ziv Welch, atau selanjutnya disebut dengan algoritma LZW merupakan algoritma kompresi yang cukup populer. Proses kompresi maupun dekompresi algoritma LZW secara spesifik akan dijabarkan pada pembahasan berikut. Kompresi algoritma LZW dimulai dengan menginisialisasi kamus dengan seluruh simbol di dalam alfabet. Jumlah keseluruhan simbol adalah 256 buah, sehingga pada mulanya kamus algoritma LZW telah terisi dengan 256 buah simbol sebelum dilakukan pembacaan pada berkas yang akan dikompresi. Hal inilah yang merupakan keunggulan utama pada algoritma LZW, yaitu karena kamus sudah diisi terlebih dahulu, maka karakter yang akan dibaca pada berkas pastilah telah terdapat di dalam kamus, sehingga token LZW hanyalah mengandung pointer karakter tersebut pada kamus saja. Akibatnya, berkas hasil kompresi akan menjadi lebih mampat dibandingkan hasil kompresi algoritma lain seperti LZ77.

Prinsip utama proses kompresi algoritma LZW adalah membaca setiap karakter satu per satu dan mengakumulasikannya ke dalam sebuah string. Selama string ini masih ditemukan di dalam kamus, maka proses pembacaan akan terus berlanjut. Ketika string tidak ditemukan di dalam kamus, sebut saja string Sx (yaitu string S dikonkatenasi dengan karakter x), maka urutan string S dalam kamus akan dituliskan oleh encoder LZW, string Sx akan ditambahkan pada kamus, dan menginisialisasi string awal sebagai karakter x.

Untuk memahami secara mendetail, diberikan sebuah contoh kasus kompresi teks “botbananaboat”. Karakter yang dibaca pertama kali ialah “b”, yaitu terdapat di dalam kamus dengan indeks 98. Selanjutnya “b” akan dikonkatenasi dengan “a”, dimana “ba” tidak terdapat di dalam kamus. Sehingga encoder LZW akan menuliskan 98, yaitu indeks karakter “b”, kemudian menambahkan “ba” di dalam kamus, dan mulai melakukan pembacaan karakter dari karakter “a”.

Proses kompresi tahap per tahap dari teks “bananaboat” ditunjukkan pada tabel berikut.

String Awal	Di dalam kamus?	Entri baru di dalam kamus	Keluaran encoder
b	Ada		
bo	Tidak	bo(257)	b
o	Ada		
ot	Tidak	ot(258)	o
t	Ada		
tb	Tidak	tb(259)	t
b	Ada		
ba	Tidak	ba(260)	b(98)
a	Ada		
an	Tidak	an(261)	a(97)
n	Ada		
na	Tidak	na(262)	n(110)
a	Ada		
an	Ada		
ana	Tidak	ana(263)	an(258)
a	Ada		
ab	Tidak	ab(264)	a(97)
b	Ada		
bo	Ada		
boa	Tidak	boa(265)	bo
a	Ada		
at	Tidak	at(266)	a(97)

t	Ada		
t,EOF	Tidak		t(116)

Maka hasil dari kompresi teks “botbananaboat” dengan algoritma LZW adalah b(98), a(97), n(110), an(258), a(97), b(98), o(111), a(97), t(116).

3.2. Kelemahan Algoritma LZW

Algoritma LZW cukup mangkus dalam melakukan kompresi data, namun algoritma LZW sendiri memiliki kelemahan, diantaranya ialah:

Waktu kompresi yang semakin lama apabila jumlah entri dalam kamus semakin besar. Kompresi algoritma LZW dilakukan dengan mengkompresi karakter per karakter, dimana setiap karakter akan dicek apakah telah berada di dalam kamus atau tidak. Waktu yang diperlukan untuk mengecek keberadaan setiap karakter ini akan semakin lama apabila entri dalam kamus semakin banyak. Hal ini dikarenakan kamus akan ditelusuri dari entri pertama hingga entri terakhir. Contohnya: apabila entri terakhir memiliki indeks 875, maka iterasi yang dibutuhkan adalah 875 kali iterasi, yang tentunya akan memakan waktu yang cukup lama.

Pada berkas dengan karakter-karakter yang tersebar acak, jumlah entri dalam kamus akan menjadi sangat besar, dikarenakan string yang dimasukkan di dalam kamus tidak memiliki keterkaitan dengan string lainnya yang telah berada di dalam kamus. Dengan kata lain, kompresi berkas malah mengakibatkan berkas hasil kompresi dapat berukuran lebih besar dibandingkan ukuran berkas sebelum dikompresi.
Berdasarkan kedua kelemahan tersebut, maka permasalahan utama dari algoritma LZW ialah jumlah entri dalam kamus yang sangat besar, sehingga solusi yang terbaik untuk menyelesaikan persoalan ini ialah mengubah bentuk berkas sebelum dikompresi menjadi lebih teratur sehingga ukuran kamus dapat diperkecil.

3.3. Solusi yang Ditawarkan

Pengubahan bentuk berkas dapat dilakukan dengan berbagai macam metode. Salah satu metode yang terkenal adalah transformasi berkas. Transformasi berkas terdapat

tiga macam, yaitu transformasi Burrows Wheeler, transformasi Star, dan transformasi LIPT.

Transformasi Burrows Wheeler

Transformasi Burrows Wheeler akan mengubah bentuk berkas masukan, sehingga karakter-karakter yang sejenis akan berada di dalam posisi berdekatan di dalam berkas. Transformasi Burrows Wheeler menggunakan sistem permutasi pada setiap karakter di dalam berkas.

Transformasi Star

Transformasi Star akan mengubah karakter-karakter tertentu di dalam berkas menjadi karakter '*'. Pengubahan bentuk karakter ini berdasarkan panjang kata di dalam berkas, semakin panjang kata maka semakin banyak karakter yang akan diubah.

Transformasi LIPT

Transformasi LIPT (Length Index Preserving Transform) akan mengubah bentuk setiap kata di dalam berkas berdasarkan posisinya di dalam kamus. Kamus yang digunakan biasanya merupakan kamus bahasa Inggris.

Pengubahan bentuk berkas tidak akan menggunakan transformasi Star dan LIPT. Alasannya ialah kedua bentuk transformasi ini tidak efisien karena memerlukan kamus tambahan di dalam proses transformasi. Kamus tambahan yang dimaksud ialah daftar seluruh kata-kata yang terdapat di dalam bahasa (sebagai contoh bahasa Inggris apabila berkas disadurkan dalam bahasa Inggris). Kesulitan dalam pembuatan kamus tambahan inilah yang menyebabkan transformasi Star dan LIPT tidak digunakan dalam pembuatan tugas akhir ini.

Transformasi Burrows Wheeler akan digunakan pada tugas akhir ini karena transformasi ini bersifat sederhana dan tidak memerlukan kamus tambahan. Transformasi Burrows Wheeler dapat mengubah berkas sehingga karakter-karakter yang sama akan diletakkan di dalam posisi yang berdekatan di dalam berkas.

3.4. Analisis Solusi

Tinjau kembali teks “bananaboat” yang telah dibahas pada subbab 1.2. Teks ini akan mengalami proses transformasi Burrows Wheeler sehingga akan menghasilkan berkas yang berbeda, misalnya menjadi:

B	B	B	A	A	A	A	N	N	O	O	T	T

Hasil dari transformasi Burrows Wheeler terhadap teks “botbananaboat” adalah “bbbaaaannoott”. Perhatikan bahwa karakter-karakter yang sama pada teks hasil transformasi akan menjadi saling berdekatan. Setelah itu teks “nnbotaaaba” akan dikompresi dengan algoritma LZW.

String Awal	Di dalam kamus?	Entri baru di dalam kamus	Keluaran encoder
b	Ada		
bb	Tidak	bb(257)	n
b	Ada		
bb	Ada		n(97)
bba	Tidak	bba(258)	
a	Ada		b(110)
aa	Tidak	aa(259)	
a	Ada		o
aa	Ada		
aaa	Tidak	aaa(260)	t
a	Ada		
an	Tidak	an(261)	t
n	Ada		
nn	Tidak	nn(262)	a
n	Ada		
no	Tidak	no(263)	
o	Ada		aa
ot	Tidak	ot(264)	
t	Ada		b
tt	Tidak	tt(265)	
t	Ada		

t, EOF	Tidak		bo

Hasil kompresi berkas “botbananaboat” akan menjadi lebih baik apabila berkas ditransformasikan dengan transformasi Burrows Wheeler. Kompresi akan semakin baik apabila jumlah huruf yang berulang semakin banyak, misalnya “maaaaaa” akan memberikan kompresi yang lebih baik daripada “maa”.

3.5. Faktor-Faktor yang Menentukan

Beberapa faktor yang menentukan tingkat keberhasilan algoritma kompresi LZW dengan transformasi berkas Burrows Wheeler antara lain:

Bentuk transformasi berkas

Secara teori berkas yang telah tertransformasi akan semakin baik hasil kompresinya, karena karakter-karakter yang berulang telah berada dalam posisi yang berdekatan. Namun, ada kalanya pola repetisi pada berkas sudah baik sehingga transformasi berkas justru dapat membuat bentuk berkas menjadi lebih rumit, yang akibatnya akan sulit untuk dikompresi.

Ukuran kamus

Ukuran kamus pada algoritma LZW juga memiliki peranan penting. Semakin besar ukuran kamus maka waktu kompresi yang dibutuhkan akan semakin lama. Oleh karena itu, transformasi Burrows Wheeler digunakan untuk memperkecil ukuran kamus, sehingga diharapkan waktu kompresi akan menjadi lebih singkat.

3.6. Analisis Perangkat Lunak

3.6.1. Deskripsi Perangkat Lunak

Perangkat lunak yang akan dibangun ialah perangkat lunak yang mampu melakukan pengujian terhadap hasil kompresi berkas berjenis teks dengan algoritma LZW. Selain itu, perangkat lunak yang akan dibangun harus mampu untuk mentransformasikan berkas berjenis teks dengan transformasi Burrows Wheeler. Perangkat lunak yang akan dibangun merupakan perangkat lunak yang sederhana, karena tujuan dari pembangunan perangkat lunak ialah untuk menguji hasil kompresi berkas sebelum dan sesudah ditransformasikan dengan transformasi Burrows Wheeler.

Berkas yang ditangani perangkat lunak adalah berkas berjenis *plaintext* (*.txt). Perangkat lunak yang akan dibangun memiliki antarmuka yang memudahkan pengguna untuk melakukan proses kompresi maupun transformasi pada berkas. Antarmuka akan dirancang menjadi sesederhana mungkin namun tepat guna.

3.6.2. Spesifikasi Kebutuhan Perangkat Lunak

Kebutuhan fungsional perangkat lunak dapat dilihat pada tabel berikut.

ID	Deskripsi Kebutuhan
SRS-F-001	Sistem mampu melakukan kompresi berkas masukan berjenis teks dengan algoritma LZW
SRS-F-002	Sistem mampu melakukan dekompresi berkas masukan hasil kompresi dengan algoritma LZW
SRS-F-003	Sistem mampu mentransformasikan berkas masukan berjenis teks dengan transformasi Burrows Wheeler
SRS-F-004	Sistem mampu mentransformasikan berkas masukan hasil transformasi dengan transformasi Burrows Wheeler
SRS-F-005	Sistem mampu melakukan kompresi berkas masukan berjenis teks dengan gabungan algoritma LZW dan Burrows Wheeler
SRS-F-006	Sistem mampu melakukan dekompresi berkas masukan berjenis teks dengan gabungan algoritma LZW dan Burrows Wheeler

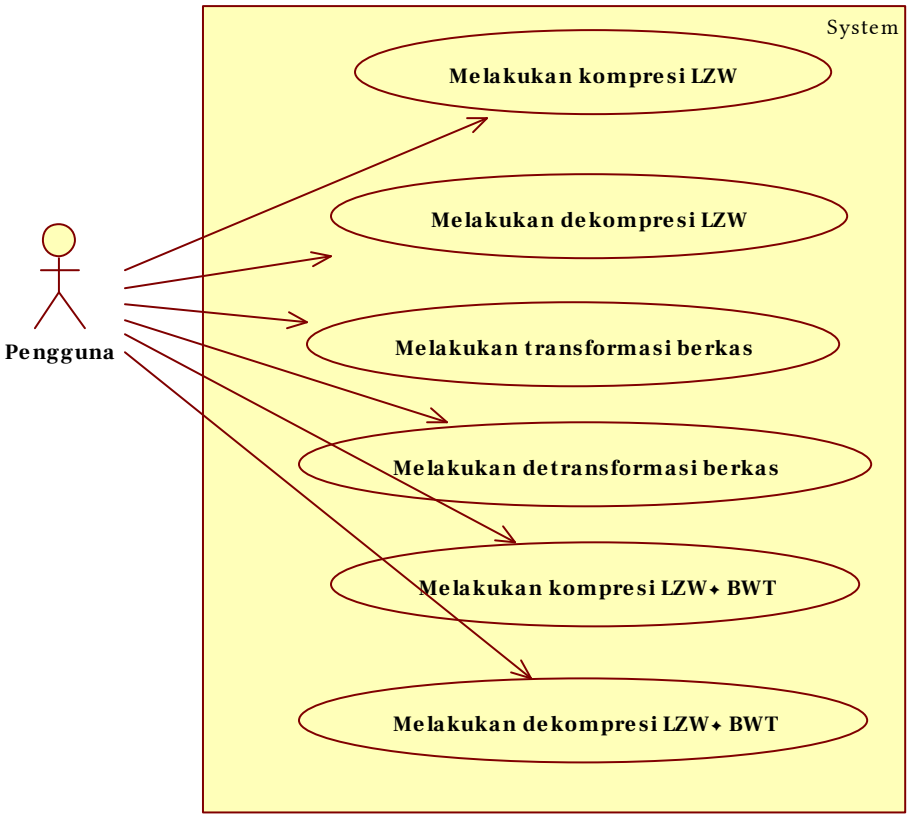
Kebutuhan non-fungsional perangkat lunak dapat dilihat pada tabel berikut

ID	Deskripsi Kebutuhan
SRS-NF-001	Sistem memiliki antarmuka yang mudah dimengerti oleh pengguna
SRS-NF-002	Sistem ditampilkan dalam bahasa Indonesia

3.6.3. Diagram Use Case

Diagram use case dari perangkat lunak dapat dilihat pada gambar berikut. Di dalam perangkat lunak ini terdapat seorang aktor, yaitu pengguna. Use case di dalam perangkat lunak ini adalah:

1. Melakukan kompresi berkas dengan algoritma LZW.
2. Melakukan dekompresi berkas dengan algoritma LZW.
3. Melakukan transformasi berkas dengan algoritma Burrows Wheeler.
4. Melakukan detransformasi berkas dengan algoritma Burrows Wheeler.
5. Melakukan kompresi berkas dengan algoritma LZW+BWT.
6. Melakukan dekompresi berkas dengan algoritma LZW+BWT.



3.6.4. Skenario Use Case

ID : UC-01

Nama use case : Melakukan kompresi berkas

Deskripsi : Pengguna dapat melakukan kompresi berkas dengan algoritma LZW

Prekondisi : Pengguna memilih berkas yang akan dikompresi

Aksi Aktor	Reaksi Sistem
1. Mengklik tombol untuk memilih berkas yang akan dikompresi	
	2. Menampilkan openfile dialog
3. Mengklik tombol untuk memilih berkas hasil kompresi	
	4. Menampilkan savefile dialog
5. Memilih algoritma LZW, memilih mode kompresi	
6. Mengklik eksekusi	
	7. Melakukan kompresi pada berkas

	dengan algoritma LZW
	8. Memberitahukan kepada pengguna bahwa proses kompresi telah selesai

ID : UC-02

Nama use case : Melakukan dekompresi berkas

Deskripsi : Pengguna dapat melakukan dekompresi berkas dengan algoritma LZW

Prekondisi : Pengguna memilih berkas yang akan didekompresi

Aksi Aktor	Reaksi Sistem
1. Mengklik tombol untuk memilih berkas yang akan dikompresi	
	2. Menampilkan openfile dialog
3. Mengklik tombol untuk memilih berkas hasil kompresi	
	4. Menampilkan savefile dialog
5. Memilih algoritma LZW, memilih mode dekompresi	
6. Mengklik eksekusi	
	7. Melakukan kompresi pada berkas dengan algoritma LZW
	8. Memberitahukan kepada pengguna bahwa proses kompresi telah selesai

ID : UC-03

Nama use case : Melakukan transformasi berkas

Deskripsi : Pengguna dapat melakukan transformasi berkas dengan algoritma Burrows Wheeler

Prekondisi : Pengguna memilih berkas yang akan ditransformasikan

Aksi Aktor	Reaksi Sistem
1. Mengklik tombol untuk memilih berkas yang akan ditransformasikan	
	2. Menampilkan openfile dialog
3. Mengklik tombol untuk memilih berkas hasil transformasi	

	4. Menampilkan savefile dialog
5. Memilih algoritma BWT, memlih mode kompresi	
6. Mengklik eksekusi	
	7. Melakukan transformasi berkas dengan algoritma Burrows Wheeler
	8. Memberitahukan kepada pengguna bahwa proses transformasi telah selesai

ID : UC-04

Nama use case : Melakukan detransformasi berkas

Deskripsi : Pengguna dapat melakukan detransformasi berkas dengan algoritma Burrows Wheeler

Prekondisi : Pengguna memilih berkas yang akan didetransformasikan

Aksi Aktor	Reaksi Sistem
1. Mengklik tombol untuk memilih berkas yang akan didetransformasikan	
	2. Menampilkan openfile dialog
3. Mengklik tombol untuk memilih berkas hasil detransformasi	
	4. Menampilkan savefile dialog
5. Memilih algoritma BWT, memilih mode dekompresi	
6. Mengklik eksekusi	
	7. Melakukan detransformasi berkas dengan algoritma Burrows Wheeler
	8. Memberitahukan kepada pengguna bahwa proses detransformasi telah selesai

ID : UC-05

Nama use case : Melakukan kompresi berkas dengan algoritma gabungan

Deskripsi : Pengguna dapat melakukan kompresi berkas dengan algoritma gabungan LZW dan Burrows Wheeler

Prekondisi : Pengguna memilih berkas yang akan dikompresi

Aksi Aktor	Reaksi Sistem
1. Mengklik tombol untuk memilih berkas yang akan dikompresi	
	2. Menampilkan openfile dialog
3. Mengklik tombol untuk memilih berkas hasil kompresi	
	4. Menampilkan savefile dialog
5. Memilih algoritma LZW + BWT, memlih mode kompresi	
6. Mengklik eksekusi	
	7. Melakukan transformasi berkas dengan algoritma Burrows Wheeler
	8. Melakukan kompresi berkas dengan algoritma LZW
	9. Memberitahukan kepada pengguna bahwa proses kompresi telah selesai

ID : UC-06

Nama use case : Melakukan dekompresi berkas dengan algoritma gabungan

Deskripsi : Pengguna dapat melakukan dekompresi berkas dengan algoritma gabungan LZW dan Burrows Wheeler

Prekondisi : Pengguna memilih berkas yang akan didekompresi

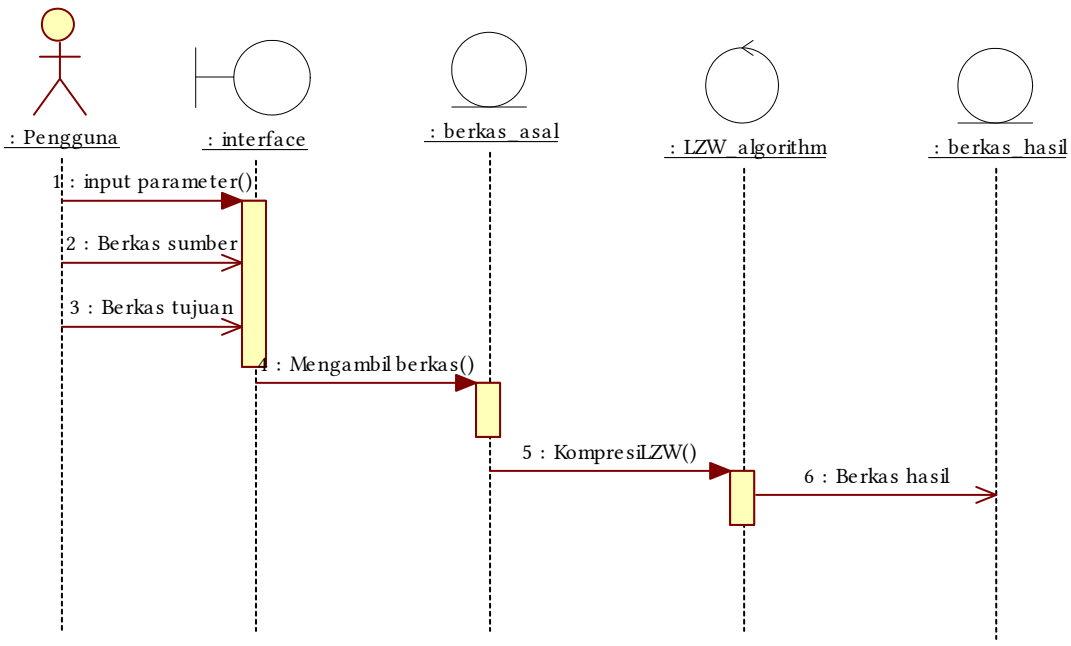
Aksi Aktor	Reaksi Sistem
1. Mengklik tombol untuk memilih berkas yang akan didekompresi	
	2. Menampilkan openfile dialog
3. Mengklik tombol untuk memilih berkas hasil dekompresi	
	4. Menampilkan savefile dialog
5. Memilih algoritma LZW + BWT,	

memlih mode dekompresi	
6. Mengklik eksekusi	
	7. Melakukan dekompresi berkas dengan algoritma LZW
	8. Melakukan detransformasi berkas dengan algoritma Burrows Wheeler
	9. Memberitahukan kepada pengguna bahwa proses dekompresi telah selesai

3.6.5. Realisasi Use Case Tahap Analisis

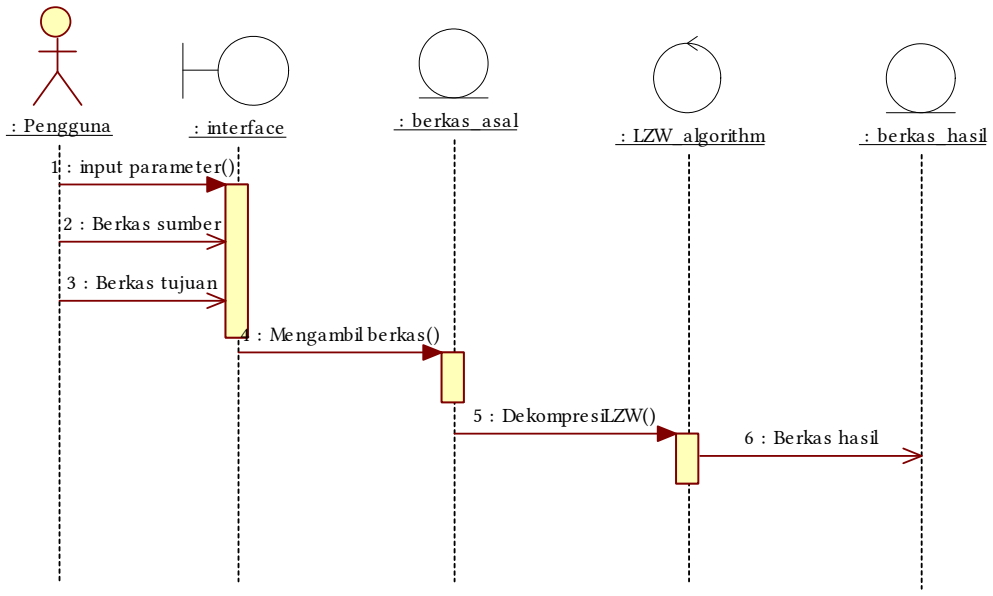
3.6.5.1. Use Case Melakukan Kompresi Berkas

Pada use case melakukan kompresi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas controller yang menangani kompresi berkas dengan algoritma LZW. Kelas berkas_asal merupakan kelas entity yang berisikan berkas yang akan dikompresi, dan kelas berkas_hasil merupakan kelas entity yang berisikan berkas yang hasil kompresi.



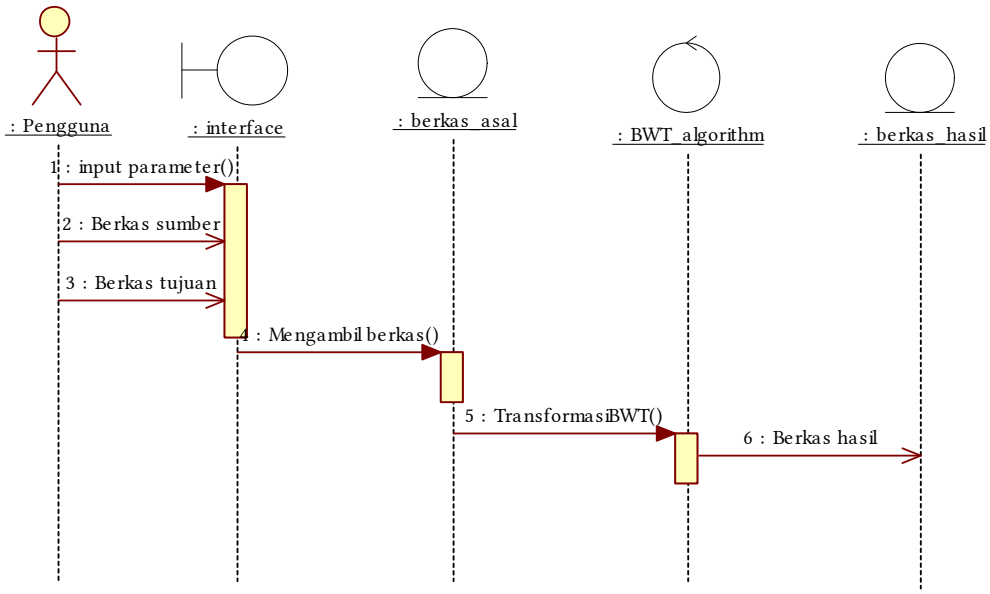
3.6.5.2. Use Case Melakukan Dekompresi Berkas

Pada use case melakukan dekompresi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas controller yang menangani dekompresi berkas dengan algoritma LZW. Kelas berkas_asal merupakan kelas entity yang berisikan berkas yang akan didekompresi, dan kelas berkas_hasil merupakan kelas entity yang berisikan berkas yang hasil dekompresi.



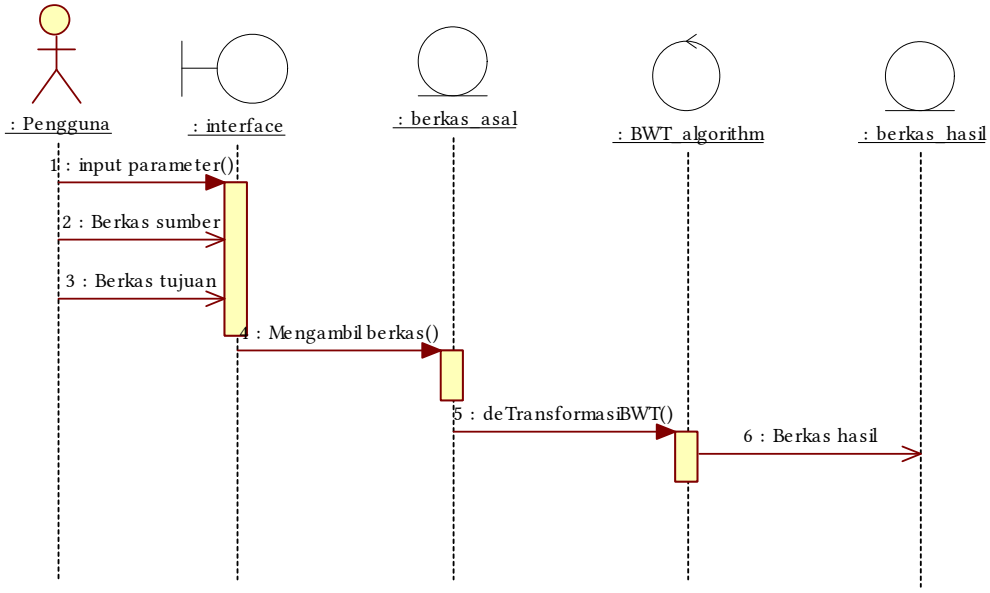
3.6.5.3. Use Case Melakukan Transformasi Berkas

Pada use case melakukan transformasi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas BWT_algorithm adalah kelas controller yang menangani transformasi berkas dengan transformasi Burrows Wheeler. Kelas berkas_asal merupakan kelas entitiy yang berisikan berkas yang akan ditransformasi, dan kelas berkas_hasil merupakan kelas entity yang berisikan berkas yang hasil transformasi.



3.6.5.4. Use Case Melakukan Detransformasi Berkas

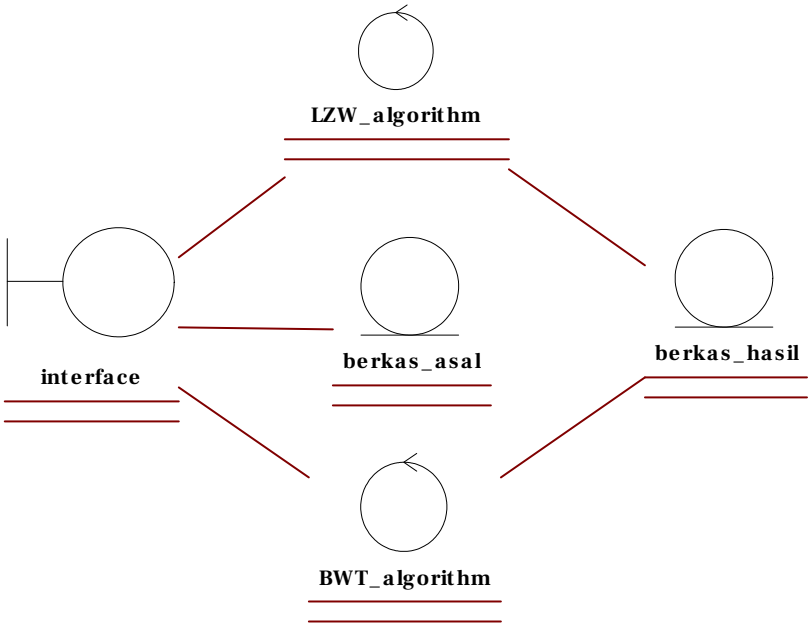
Pada use case melakukan detransformasi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas BWT_algorithm adalah kelas controller yang menangani detransformasi berkas dengan transformasi Burrows Wheeler. Kelas berkas_asal merupakan kelas entitiy yang berisikan berkas yang akan didetransformasi, dan kelas berkas_hasil merupakan kelas entity yang berisikan berkas yang hasil detransformasi.



3.6.6. Diagram Kelas Analisis

Diagram kelas analisis perangkat lunak dapat dilihat pada gambar berikut. Kelas-kelas yang terlibat antara lain:

1. Kelas interface, adalah kelas *boundary* yang merupakan antarmuka untuk aktor pengguna.
2. Kelas BWTAlgorithm, adalah kelas *controller* yang merupakan kelas yang menangani proses transformasi dan detransformasi Burrows Wheeler pada berkas.
3. Kelas LZWAlgorithm, adalah kelas *controller* yang merupakan kelas yang menangani proses kompresi dan dekompresi LZW pada berkas.
4. Kelas berkas_asal, adalah kelas *entitiy* yang merupakan berkas yang akan dikompresi, didekompresi, ditransformasi, dan didetransformasi.
5. Kelas berkas_hasil, adalah kelas *entity* yang merupakan berkas hasil kompresi, dekompresi, transformasi, maupun detransformasi.



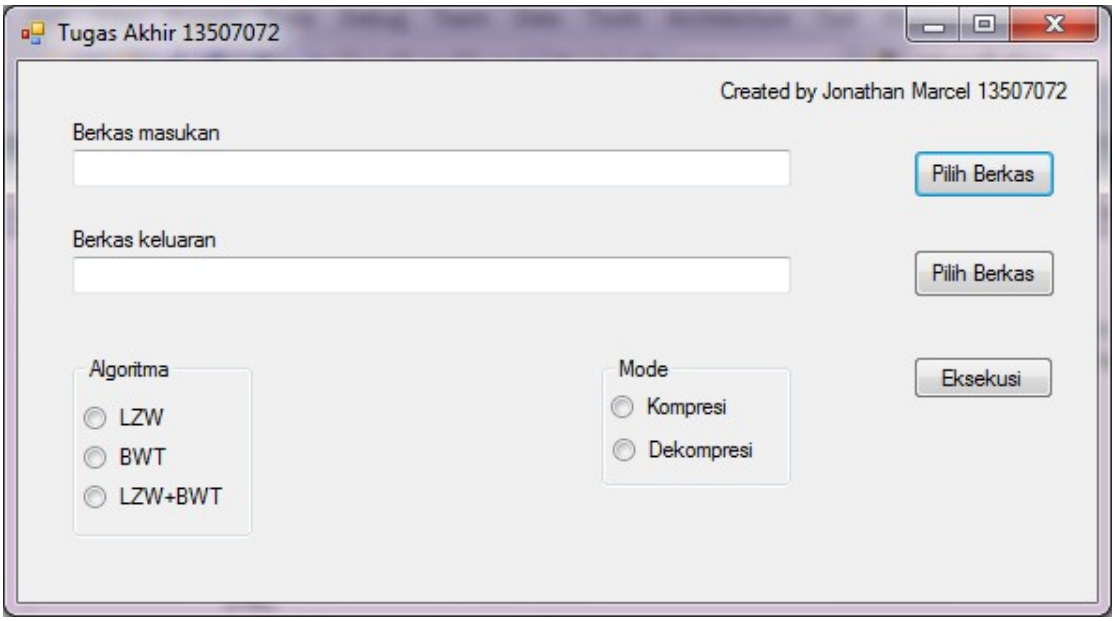
BAB IV
PERANCANGAN, IMPLEMENTASI, DAN EVALUASI

Pada bab ini, akan dijelaskan hal-hal yang meliputi perancangan, implementasi, dan evaluasi kinerja perangkat lunak. Pada bagian perancangan, akan dijelaskan mengenai diagram kelas pada tahap perancangan, perancangan antarmuka, dan realisasi *use case* pada tahap perancangan. Pada bagian implementasi, akan dijelaskan mengenai spesifikasi perangkat keras maupun perangkat lunak. Pada bagian evaluasi, akan dijelaskan mengenai evaluasi yang dilakukan dan analisis hasil evaluasi. Evaluasi yang dilakukan antara lain evaluasi perangkat lunak dan evaluasi algoritma.

4.1 Perancangan

4.1.1. Perancangan Antarmuka

Perangkat lunak akan memiliki tampilan seperti pada gambar berikut. Pengguna dapat memilih berkas asal yang diinginkan dan dapat menyimpan berkas hasil dengan nama berkas yang diinginkan. Pengguna juga dapat memilih pilihan algoritma yang akan digunakan, yaitu kompresi, dekompresi, transformasi, dan detransformasi, dengan mengklik tombol yang sesuai.



Setelah proses kompresi, dekompresi, transformasi, dan detransformasi selesai, sistem akan menampilkan *message box* yang berisikan informasi bahwa proses telah selesai.

4.1.2. Realisasi Use Case Tahap Perancangan

Use Case Melakukan Kompresi Berkas

Pada use case melakukan kompresi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas controller yang menangani kompresi berkas dengan algoritma LZW. Kelas berkas_asal merupakan kelas entity yang berisikan berkas yang akan dikompresi, dan kelas berkas_hasil merupakan kelas entity yang berisikan berkas yang hasil kompresi.

Use Case Melakukan Dekompresi Berkas

Pada use case melakukan dekompresi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas controller yang menangani dekompresi berkas dengan algoritma LZW. Kelas berkas_asal merupakan kelas entity yang berisikan berkas yang akan didekompresi, dan kelas berkas_hasil merupakan kelas entity yang berisikan berkas yang hasil dekompresi.

Use Case Melakukan Transformasi Berkas

Pada use case melakukan kompresi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas BWT_algorithm adalah kelas controller yang menangani transformasi berkas dengan transformasi Burrows Wheeler. Kelas berkas_asal merupakan kelas entitiy yang berisikan berkas yang akan ditransformasi, dan kelas berkas_hasil merupakan kelas entity yang berisikan berkas yang hasil transformasi.

Use Case Melakukan Detransformasi berkas

Pada use case melakukan detransformasi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas BWT_algorithm adalah kelas controller yang menangani detransformasi berkas dengan transformasi Burrows Wheeler. Kelas berkas_asal merupakan kelas entitiy yang berisikan berkas yang akan didetransformasi, dan kelas berkas_hasil merupakan kelas entity yang berisikan berkas yang hasil detransformasi.

4.1.3. Diagram Kelas Perancangan

Sesuai dengan diagram kelas hasil analisis, terdapat lima buah kelas yang terlibat di dalam model perancangan. Diagram kelas perancangan dapat dilihat pada gambar berikut. Kelas-kelas yang terlibat antara lain:

1. Kelas interface, adalah *boundary class* yang merupakan antarmuka untuk aktor pengguna.
2. Kelas BWTAlgorithm, adalah *controller class* yang merupakan kelas yang menangani proses transformasi dan detransformasi Burrows Wheeler pada berkas.
3. Kelas LZWAlgorithm, adalah *controller class* yang merupakan kelas yang menangani proses kompresi dan dekompresi LZW pada berkas.
4. Kelas berkas_asal, adalah *entity class* yang merupakan berkas yang akan dikompresi, didekompresi, ditransformasi, dan didetransformasi.
5. Kelas berkas_hasil, adalah *entity class* yang merupakan berkas hasil kompresi, dekompresi, transformasi, maupun detransformasi.

4.2. Implementasi

4.2.1. Lingkungan Implementasi

Perangkat lunak dikembangkan pada perangkat keras berbasis *desktop* dengan spesifikasi sebagai berikut:

1. Prosesor Intel Core i5 2,26 GHz
2. Memory RAM 2GB
3. Harddisk 500GB
4. Perangkat masukan *keyboard* dan *mouse*

Adapun perangkat lunak yang digunakan memiliki spesifikasi sebagai berikut:

1. Sistem operasi Microsoft Windows 7
2. Kakas pengembang Microsoft Visual Studio 2010 dengan bahasa pemrograman C#
3. Notepad++

4.2.2. Batasan Implementasi

Perangkat lunak hanya akan memroses berkas berjenis *plain text* dengan ekstensi (*.txt).

4.3 Evaluasi

4.3.1 Evaluasi Perangkat Lunak

Evaluasi dilakukan untuk mengevaluasi kebenaran perangkat lunak yang mencakup proses kompresi dan dekompresi berkas. Pada evaluasi ini akan dilihat apakah perangkat lunak berhasil mengkompresi maupun mendekompresi berkas sehingga menjadi kembali ke berkas awal dengan sempurna.

4.3.1.1. Lingkungan Evaluasi

Evaluasi digunakan pada perangkat keras berbasis *desktop* dengan spesifikasi sebagai berikut:

1. Prosesor Intel Core i5 2,26 GHz
2. Memory RAM 2GB

3. Harddisk 500GB
4. Perangkat masukan *keyboard* dan *mouse*

4.3.1.2. Tujuan Evaluasi

Tujuan dari evaluasi perangkat lunak ini adalah:

1. Menguji kebenaran proses kompresi maupun dekompresi berkas dengan algoritma Lempel-Ziv-Welch
2. Menguji kebenaran proses transformasi dan detransformasi berkas dengan algoritma Burrows Wheeler
3. Menguji kebenaran proses kompresi maupun dekompresi berkas dengan berkas yang telah dimanipulasi dengan transformasi Burrows Wheeler

4.3.1.3. Data Uji

Data uji yang digunakan ialah berupa berkas teks yang terdiri atas berkas-berkas sebagai berikut.

No.	Nama Berkas	Deskripsi	Ukuran (bytes)
1	red_dead_redemption_c.txt	Panduan game “Red Dead Redemption”	335064
2	1musk10.txt	E-book berjudul: “The Three Musketeers”	1344739
3	anne11.txt	E-book berjudul: “Anne of Green Gables”	587053

4.3.1.4. Kasus Uji dan Hasil Evaluasi

Berdasarkan tujuan evaluasi yang telah dijabarkan pada subbab 4.3.1.2., maka didefinisikan tiga buah kasus uji untuk mengevaluasi kebenaran perangkat lunak.

1. Kasus uji pertama: Menguji kebenaran proses kompresi dan dekompresi berkas dengan algoritma LZW. Pengujian dinyatakan diterima apabila berkas hasil kompresi dapat dikompresi dan tidak memiliki perbedaan dengan berkas awal sebelum dikompresi. Perbandingan berkas dilakukan dengan kakas Notepad++ dengan *Plugin Compare*.

Berkas Masukan	Berkas Keluaran	Ukuran	Ukuran	Binary	Kesimpulan

		Masukan (byte)	Keluaran (byte)	Compare	
red_dead_redemption_c.txt	red_dead_redemption_c(com).txt	335064	164238	Files Match!	Diterima
red_dead_redemption_c(com).txt	red_dead_redemption_c(decom).txt	164238	335064		Diterima
1musk10.txt	1musk10(com).txt	1344739	750028	Files Match!	Diterima
1musk10(com).txt	1musk10(decom).txt	750028	1344739		Diterima
anne11.txt	anne11(com).txt	587053	331543	Files Match!	Diterima
anne11(com).txt	anne11(decom).txt	331543	587053		Diterima

2. Kasus uji kedua: Menguji kebenaran proses transformasi dan detransformasi berkas dengan algoritma Burrows Wheeler. Pengujian dinyatakan diterima apabila berkas hasil kompresi dapat dikompresi dan tidak memiliki perbedaan dengan berkas awal sebelum dikompresi. Perbandingan berkas dilakukan dengan kakas Notepad++ dengan *Plugin Compare*.

Berkas Masukan	Berkas Keluaran	Ukuran Masukan (byte)	Ukuran Keluaran (byte)	Binary Compare	Kesimpulan
red_dead_redemption_c.txt	red_dead_redemption_c(t).txt	335064	335072	Files Match!	Diterima
red_dead_redemption_c(t).txt	red_dead_redemption_c(dt).txt	335072	335064		Diterima
1musk10.txt	1musk10(t).txt	1344739	750028	Files Match!	Diterima
1musk10(t).txt	1musk10(dt).txt	750028	1344739		Diterima
anne11.txt	anne11(t).txt	587053	587065	Files Match!	Diterima
anne11(t).txt	anne11(dt).txt	587065	587053		Diterima

3. Kasus uji ketiga: Menguji kebenaran proses kompresi dan dekompresi berkas dengan algoritma LZW, dimana sebelum dikompresi berkas telah ditransformasikan dengan algoritma Burrows Wheeler. Pengujian dinyatakan diterima apabila berkas hasil kompresi dapat dikompresi dan setelah mengalami detransformasi tidak memiliki perbedaan dengan berkas awal sebelum dikompresi. Perbandingan berkas dilakukan dengan kakas Notepad++ dengan *Plugin Compare*.

Berkas Masukan	Berkas Keluaran	Ukuran Masukan (byte)	Ukuran Keluaran (byte)	Binary Compare	Kesimpulan
red_dead_redemption_c.txt	red_dead_redemption_c(t).txt	335064	335072	Files Match!	Diterima
red_dead_redemption_c(t).txt	red_dead_redemption_c(dt).txt	335072	335064		Diterima
1musk10.txt	1musk10(t).txt	1344739	750028	Files Match!	Diterima
1musk10(t).txt	1musk10(dt).txt	750028	1344739		Diterima
anne11.txt	anne11(t).txt	587053	587065	Files Match!	Diterima
anne11(t).txt	anne11(dt).txt	587065	587053		Diterima

4.4 Kesimpulan Hasil Pengujian

Dari hasil pengujian skema, tampak bahwa *schema aware* merupakan skema yang memiliki waktu eksekusi *query* paling singkat untuk sebagian besar operasi yang mungkin terjadi dalam penggunaan kamus bahasa. Di sisi lain, performansi skema *hybrid* menempati posisi kedua terbaik, dan *schema oblivious* merupakan skema terburuk. Namun terdapat sebuah kekurangan fatal yang menyebabkan *schema aware* tidak dipilih sebagai skema yang akan diimplementasikan, yaitu lamanya operasi *retrieve* untuk sebuah entri.

Seorang pengguna kamus bahasa (*end-user*) akan sangat sering melakukan akses terhadap sebuah entri tertentu untuk melihat semua properti yang terkait dengan entri tersebut. Sedangkan pada *schema aware* pengambilan detail dari sebuah entri akan

membutuhkan akses ke sejumlah n tabel (n = jumlah properti/predikat dari entri). Selain itu, untuk penggunaan oleh *linguist*, pengelolaan basis data akan menjadi lebih rumit dengan *schema aware* (banyaknya tabel yang harus dikelola). Sementara itu, skema *hybrid* memberikan performansi yang cukup baik untuk semua kasus uji. Oleh karena itu, dalam implementasinya, dipilih skema *hybrid* sebagai skema untuk menyimpan data kamus bahasa.

Dari hasil pengujian, tampak bahwa beban *server* dapat dikurangi dengan membagi tempat pemrosesan *query* ke komputer yang berbeda. Oleh karena itu, konsep basis data terdistribusi dengan fragmentasi horizontal dan replikasi data bahasa Indonesia dapat diaplikasikan untuk penelitian tugas akhir ini, dengan tujuan meningkatkan performansi sistem.

Sebagai kesimpulan, dapat diambil beberapa poin penting terkait hasil pengujian:

- Meskipun *schema aware* memberikan hasil terbaik untuk hampir semua kasus, skema ini tidak dapat digunakan karena tidak sesuai dengan kebutuhan penggunaan kamus
- Skema *hybrid* menunjukkan performansi yang baik untuk semua kasus uji, sehingga skema ini cocok untuk kasus kamus bahasa.
- Untuk lebih meningkatkan performansi sistem, data dapat dialokasikan secara terdistribusi ke beberapa *server*, sehingga beban *server* tidak terakumulasi pada satu *site* saja.

BAB V
KESIMPULAN DAN SARAN

Bab ini berisi beberapa kesimpulan dan saran yang diperoleh dari penelitian tugas akhir ini.

5.1 Kesimpulan

Kesimpulan yang didapatkan dari penelitian ini adalah sebagai berikut:

1. RDF dan ontologi sebagai bagian dari web semantik merupakan standar yang digunakan untuk representasi data dalam web sehingga tidak hanya sekedar dapat dibaca, tetapi juga dapat dipahami oleh mesin
2. *Library* ARC tidak cocok jika digunakan untuk menyimpan data kamus multibahasa
3. Dari ketiga representasi yang ada, skema *hybrid* merupakan skema yang paling cocok untuk menyimpan data kamus bahasa
4. Sistem dengan rancangan skema basis data baru memberikan performansi yang lebih baik dibandingkan sistem sebelumnya
5. Penyimpanan data secara terdistribusi meningkatkan performansi sistem dengan membagi beban ke beberapa *server*

5.2 Saran

Saran yang dapat disampaikan untuk penelitian lebih lanjut adalah sebagai berikut:

1. Kombinasi strategi penyimpanan yang diimplementasikan dengan strategi pencarian yang dikembangkan oleh rekan yang lain akan meningkatkan performansi lebih jauh lagi
2. Pengembangan rancangan skema basis data sehingga dapat digunakan untuk menyimpan ontologi bahasa
3. Penelitian dapat dilakukan lebih lanjut dengan menggunakan basis data nonrelasional, seperti *XML database* atau *Graph database*, yang saat ini banyak digunakan untuk aplikasi berbasis web semantik

DAFTAR REFERENSI

[HAN03] Hankerson, Darrel. (2003). Introduction to Information Theory and Data Compression 2nd Edition. CRC.

[RAD09] Radescu, Radu. (2009). Transform Methods Used In Lossless Compression of Text Files. University of Bucharest.

[SAL07] Salomon, David. (2007). Data Compression: The Complete Reference 4th Edition. Springer-Verlag.

[SHA48] Shannon, C.E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal vol 27.

[SIM08] Simpson, Jamie. (2008). Words with Simple Burrows-Wheeler Transform. Curtin University of Technology.

[WIT99] Witten, Ian H. (1999). Managing Gigabyte: Compressing and Indexing Document and Images. Kaufmann Publisher.

[ZIV77] Ziv, Lempel. (1997). A Universal Algorithm For Sequential data Compression. IEEE.

LAMPIRAN A

QUERY UNTUK PENGUJIAN

A.1 Perbandingan Ukuran Repositori

A.1.1 Ukuran Basis Data secara Keseluruhan

Query yang digunakan adalah sebagai berikut:

```
SELECT s.schema_name,
 CONCAT(IFNULL(ROUND(SUM(t.data_length)/1024/1024,2),0.00),"Mb") AS Data_size,
 CONCAT(IFNULL(ROUND(SUM(t.index_length)/1024/1024,2),0.00),"Mb") AS Index_size,
 COUNT(table_name) AS total_tables
FROM
 INFORMATION_SCHEMA.SCHEMATA s LEFT JOIN INFORMATION_SCHEMA.TABLES t ON
  s.schema_name = t.table_schema
WHERE s.schema_name in
 ("langitschemaoblivious","langitschemaaware","langithybrid")
GROUP BY s.schema_name
ORDER BY Data_size DESC;
```

A.1.2 Ukuran Tiap Tabel dalam Basis Data

Query yang digunakan adalah sebagai berikut:

```
SELECT s.schema_name,
 t.table_name,
 CONCAT(IFNULL(ROUND(SUM(t.data_length)/1024/1024,2),0.00),"Mb")
  AS data_size,
 CONCAT(IFNULL(ROUND(SUM(t.index_length)/1024/1024,2),0.00),"Mb")
  AS index_size,
 t.ENGINE ENGINE,
 t.table_rows TABLE_ROWS,
 t.row_format TABLE_ROW_FORMAT,
 date(t.update_time)
FROM INFORMATION_SCHEMA.SCHEMATA s LEFT JOIN INFORMATION_SCHEMA.TABLES t
 ON s.schema_name = t.table_schema
WHERE s.schema_name in
 ("langitschemaoblivious","langitschemaaware","langithybrid")
GROUP BY s.schema_name,t.table_name,TABLE_ROW_FORMAT,ENGINE
ORDER BY TABLE_ROWS DESC,data_size DESC,index_size DESC;
```

A.2 Pengujian Skema

A.2.1 *Retrieve*

1. Triplet (*resource*)

   a. *Schema Oblivious*

```
SELECT *
FROM sunda_triples TR JOIN sunda_id_to_val V1 ON (V1.id =
TR.s)
  JOIN sunda_id_to_val V2 ON (V2.id = TR.p)
  JOIN sunda_id_to_val V3 ON (V3.id = TR.o)
WHERE V1.val LIKE
http://langit.itb.ac.id/dictionary/sunda/sunda_22'
  AND V2.val LIKE
'http://langit.itb.ac.id/dictionary/sunda/sunda_9'
  AND V3.val LIKE
    'http://langit.itb.ac.id/dictionary/sunda/sunda_100050'
```

   b. *Schema Aware*

```
SELECT V1.val AS s, V2.val AS o
FROM sunda_p9 TR JOIN sunda_id_to_val V1 ON ( TR.s =
V1.id )
  JOIN sunda_id_to_val V2 ON ( V2.id = TR.o )
WHERE V1.val LIKE 'sunda:sunda_22'
  AND V2.val LIKE 'sunda:sunda_100050'
```

   c. *Hybrid*

```
SELECT *
FROM sunda_triple_resource TR JOIN sunda_id_to_val V1
  ON (V1.id = TR.s) JOIN sunda_id_to_val V2 ON (V2.id = TR.p)
  JOIN sunda_id_to_val V3 ON (V3.id = TR.o)
WHERE V1.val LIKE 'sunda:sunda_22'
  AND V2.val LIKE 'sunda:sunda_9'
  AND V3.val LIKE 'sunda:sunda_100050'
```

2. Triplet (literal)

   a. *Schema Oblivious*

```
SELECT *
FROM sunda_triples TR JOIN sunda_id_to_val V1 ON (V1.id =
TR.s)
  JOIN sunda_id_to_val V2 ON (V2.id = TR.p)
  JOIN sunda_id_to_val V3 ON (V3.id = TR.o)
WHERE V1.val LIKE
 'http://langit.itb.ac.id/dictionary/sunda/sunda_100020'
 AND V2.val LIKE
 'http://langit.itb.ac.id/dictionary/sunda/sunda_20'
 AND V3.val LIKE
 'http://langit.itb.ac.id/dictionary/sunda/sunda_literal_210
0021'
```

b. *Schema Aware*

```
SELECT V1.val AS s, TR.o AS o
FROM sunda_p20 TR JOIN sunda_id_to_val V1 ON ( TR.s =
V1.id )
WHERE V1.val LIKE 'sunda:sunda_100020'
  AND TR.o LIKE 'sunda_literal_2100021'
```

c. *Hybrid*

```
SELECT *
FROM sunda_triple_literal TR JOIN sunda_id_to_val V1 ON
(V1.id=TR.s)
  JOIN sunda_id_to_val V2 ON (V2.id=TR.p)
WHERE V1.val LIKE 'sunda:sunda_100020'
  AND V2.val LIKE 'sunda:sunda_20'
  AND TR.o LIKE 'sunda_literal_2100021'
```

3. Entri

a. *Schema Oblivious*

```
SELECT V2.val AS p, V3.val AS o
FROM jawa_triples TR JOIN jawa_id_to_val V1 ON ( TR.s =
V1.id )
  JOIN ".$lang."_id_to_val V2 ON ( V2.id = TR.p )
  JOIN ".$lang."_id_to_val V3 ON ( V3.id = TR.o )
WHERE V1.val LIKE
  'http://langit.itb.ac.id/dictionary/jawa/jawa_25035'
```

b. *Schema Aware*

```
SELECT *
FROM jawa_p1 TR JOIN jawa_id_to_val V1 ON ( TR.s = V1.id )
WHERE V1.val LIKE 'jawa:jawa_25035'

UNION ALL

SELECT *
FROM jawa_p2 TR JOIN jawa_id_to_val V1 ON ( TR.s = V1.id )
WHERE V1.val LIKE 'jawa:jawa_25035'

UNION ALL

SELECT *
FROM jawa_p3 TR JOIN jawa_id_to_val V1 ON ( TR.s = V1.id )
WHERE V1.val LIKE 'jawa:jawa_25035'

UNION ALL

...

SELECT *
FROM jawa_p19 TR JOIN jawa_id_to_val V1 ON ( TR.s = V1.id )
WHERE V1.val LIKE 'jawa:jawa_25035'
```

c. *Hybrid*

```
--query resource--
```

```
SELECT *
FROM jawa_triple_resource TR JOIN jawa_id_to_val V1 ON
(TR.s=V1.id)
WHERE V1.val LIKE 'jawa:jawa_25035'

--query literal–
"SELECT *
FROM jawa_triple_literal TR JOIN jawa_id_to_val V1 ON
(TR.s=V1.id) WHERE V1.val LIKE 'jawa:jawa_25035'
```

4. Bahasa (semua entri dalam 1 bahasa)

   Untuk *retrieve* semua entri dalam 1 bahasa, ketiga skema (baik *schema oblivious*, *schema aware*, maupun *hybrid*) menggunakan *query* yang sama, yaitu:

```
SELECT * FROM jawa_id_to_val
```

A.2.2 *Insert*

Pada dasarnya *insert* suatu triplet dilakukan dengan memeriksa terlebih dahulu apakah subjek, predikat, dan objek yang akan ditambahkan ke basis data sudah ada sebelumnya. Pengecekan dilakukan sambil me-*retrieve* ID dari tabel [bahasa_daerah]_id_to_val. Jika ID ditemukan, berarti kata tersebut sudah pernah diisikan ke basis data, sehingga ID dapat langsung digunakan untuk menambahkan triplet ke tabel yang bersesuaian. Jika ID tidak ditemukan, maka perlu dilakukan penambahan *record* ke tabel [bahasa_daerah]_id_to_val sebelum menambahkan triplet, dengan *query* sebagai berikut (contoh: entri bahasa Jawa):

```
INSERT INTO jawa_id_to_val (val, id_indonesia) VALUES
  ('http://langit.itb.ac.id/dictionary/jawa/jawa_23', 1)
```

Selanjutnya, penambahan triplet dilakukan dengan:

1. *Schema Oblivious*

```
INSERT INTO jawa_triples (s, p, o) VALUES ([s_id], [p_id],
[$o_id])
```

2. *Schema Aware*

   Khusus untuk *schema aware*, pengecekan predikat tidak dilakukan ke tabel [bahasa_daerah]_id_to_val, melainkan dengan mendaftar semua tabel dalam basis data. Jika tidak terdapat tabel predikat yang diinginkan, perlu dibuat

sebuah tabel baru untuk predikat tersebut. *Query* yang digunakan adalah sebagai berikut:

```
CREATE TABLE IF NOT EXISTS `jawa_newP` (
  `s` int(11) NOT NULL,
  `o` int(11) NOT NULL,
  PRIMARY KEY (`s`, `o`),
  FOREIGN KEY (`s`) REFERENCES jawa_id_to_val (`id`) ON UPDATE
CASCADE,
  FOREIGN KEY (`o`) REFERENCES jawa_id_to_val (`id`) ON UPDATE
CASCADE
)
```

Setelah tabel baru terbentuk, kemudian barulah triplet ditambahkan ke tabel tersebut:

```
INSERT INTO jawa_newP VALUES ([s_id], [o_id])
```

3. *Hybrid*

   Operasi *insert* pada skema *hybrid* mirip dengan *insert* pada *schema oblivious*, hanya saja perlu penyesuaian tabel tempat terjadi penambahan *record*, apakah objek bertipe *resource* atau literal.

A.2.3 *Update*

1. *Update by* P

   a. *Schema Oblivious*

   ```
   UPDATE sunda_triples SET p=[p_id] WHERE s=[s_id] AND
   o=[o_id]
   ```

   b. *Schema Aware*

   Apabila hendak ditambahkan predikat baru, perlu dibuat tabel dengan cara yang sama dengan operasi *insert* pada bagian A.2.2. Setelah tabel tersedia, *update* dilakukan dengan *query* sebagai berikut:

   ```
   INSERT INTO sunda_[newP] (s, o) VALUES ([s_id], [o_id])
   ```

   c. *Hybrid*

   Serupa dengan operasi *insert*, operasi *update* pada skema *hybrid* mirip dengan *update* pada *schema oblivious*, hanya saja perlu penyesuaian tabel tempat terjadi penambahan *record*, apakah objek bertipe *resource* atau literal.

2. *Update by* O

   a. *Schema Oblivious*

      Operasi *update* objek pada *schema oblivious* sama dengan *update* predikat.

   b. *Schema Aware*

```
UPDATE sunda_[p] SET o=[o_id] WHERE s=[s_id]
```

   c. *Hybrid*

      Idem 1.c.

A.2.4 Terjemahan

Operasi terjemahan pada ketiga skema menggunakan mekanisme yang sama, karena tabel yang diakses hanyalah tabel [bahasa_daerah]_id_to_val dan indonesia_id_to_val. Adapun *query* yang digunakan adalah sebagai berikut:

1. Bahasa Indonesia ke bahasa daerah

```
SELECT ltitv.val
FROM indonesia_id_to_val iitv INNER JOIN [bhs_daerah]_id_to_val
ltitv
  ON (iitv.id = ltitv.id_indonesia)
WHERE iitv.id = [s_id]
```

2. Bahasa Jawa ke bahasa Sunda

   Untuk *single server*:

```
SELECT ltitv.val
FROM jawa_id_to_val lfitv INNER JOIN sunda_id_to_val ltitv
  ON (lfitv.id_indonesia = ltitv.id_indonesia)
WHERE lfitv.id = [s_id]
```

   Untuk *distributed server*:

```
--1. get id_indonesia from entri_jawa (site jawa)
SELECT * FROM jawa_id_to_val WHERE val = 'jawa:jawa_27692'

--2. get entri_sunda where id_indonesia = result from 1. (site
sunda)
SELECT * FROM sunda_id_to_val WHERE id_indonesia =
[id_indonesia]
```

LAMPIRAN B
HASIL PENGUJIAN SERVER TUNGGAL DAN TERDISTRIBUSI

B.1 Pengujian Server Tunggal dan Terdistribusi

Satuan waktu yang digunakan adalah milidetik.

B.1.1 Hasil Pengujian untuk *Query* 1 Triplet

SINGLE SERVER

Jml User	Jawa	Sunda	Total
1	1955.38695	1215.70096	1585.54396
2	1240.40697	1250.16988	1245.28843
3	1444.06085	1439.19474	1441.62779
4	1566.39421	1402.36964	1484.38192
5	2448.57705	2436.52833	2442.55269
6	2916.70298	2920.17174	2918.43736
7	3360.37082	3370.4087	3365.38976
8	3932.69809	3859.36376	3896.03093
9	4344.61425	4328.63588	4336.62506
10	4810.21648	4776.28869	4793.25258
11	5281.921	5234.91858	5258.41979
12	5774.64648	5722.19243	5748.41946
13	6201.29486	6110.31348	6155.80417
14	6605.93163	6551.89456	6578.91309
15	7045.57514	6966.30124	7005.93819
16	7457.28518	7412.92191	7435.10354
17	8063.36699	7931.83523	7997.60111
18	8493.11938	8347.04754	8420.08346
19	8938.06288	8776.21079	8857.13684
20	9452.10524	9288.16093	9370.13309
21	9894.73616	9759.93628	9827.33622
22	10372.36647	10153.69298	10263.02972
23	10579.46807	10436.06188	10507.76498
24	11018.21731	10961.83198	10990.02465
25	11482.07869	11478.89912	11480.48891
26	12027.42746	11910.93705	11969.18226

DISTRIBUTED SERVER

Jml User	Jawa	Sunda	Total
1	1219.32858	1222.83587	1221.08222
2	1261.2074	1273.0349	1267.12115
3	1300.21055	1309.04236	1304.62645
4	1385.09195	1383.72094	1384.40645
5	1538.69153	1546.76573	1542.72863
6	1780.67211	1764.21632	1772.44422
7	2079.09552	1907.31288	1993.2042
8	1827.78671	2347.89485	2087.84078
9	2356.15125	2511.15619	2433.65372
10	2351.90223	3057.00593	2704.45408
11	3073.47549	2712.5196	2892.99755
12	3205.27414	3021.63165	3113.4529
13	3029.01492	3615.13983	3322.07737
14	2764.07767	4030.5158	3397.29674
15	4021.96108	3516.32026	3769.14067
16	3101.29147	4827.57127	3964.43137
17	3746.37895	4779.15749	4262.76822
18	5881.11874	3100.79111	4490.95492
19	4286.76861	4861.39005	4574.07933
20	4055.78485	5472.86422	4764.32453
21	4743.69019	5380.18417	5061.93718
22	4948.35545	5496.18553	5222.27049
23	3301.75494	8115.92115	5708.83805
24	4536.62446	6825.17003	5680.89724
25	4437.89566	7337.65014	5887.7729
26	4986.40748	7189.75092	6088.0792

27	12409.83545	12264.09328	12336.96437
28	12762.39982	12753.44787	12757.92384
29	13178.25245	13100.12538	13139.18891
30	14560.18349	14343.07429	14451.62889
31	15018.3246	14726.5544	14872.4395
32	15584.91815	15224.50703	15404.71259
33	15919.60728	15652.96167	15786.28447
34	16490.04143	16127.98626	16309.01384
35	16864.23696	16625.97274	16745.10485
36	17332.41723	17044.00172	17188.20947
37	17823.064	17531.8993	17677.48165
38	18299.15372	17846.21756	18072.68564
39	18659.4013	18276.86768	18468.13449
40	19216.85426	18932.13253	19074.4934
41	19615.84188	19344.07039	19479.95613
42	20090.5507	19737.22192	19913.88631
43	20686.44387	20124.93803	20405.69095
44	21093.50565	20640.91811	20867.21188
45	21477.8733	21143.2318	21310.55255
46	22027.49021	21442.12911	21734.80966
47	22440.34756	21973.95175	22207.14966
48	22842.01207	22509.66684	22675.83946
49	23365.13486	23012.53971	23188.83728
50	23747.30214	23372.3362	23559.81917

27	9160.05341	3904.23808	6532.14574
28	7750.2632	5268.20297	6509.23308
29	8032.42973	5724.58918	6878.50945
30	8448.63505	5508.89638	6978.76571
31	7082.89316	7493.1268	7288.00998
32	9299.92285	5895.09767	7597.51026
33	10177.30315	5102.77015	7640.03665
34	10086.30081	5927.75517	8007.02799
35	6603.19821	9540.83512	8072.01667
36	6264.81349	10416.04628	8340.42989
37	8690.18041	8319.06447	8504.62244
38	6342.56321	11657.1755	8999.86935
39	10369.70716	7540.68917	8955.19816
40	7887.54634	10297.96667	9092.7565
41	13262.89019	5845.14741	9554.0188
42	10725.62367	8696.24898	9710.93633
43	13180.772	6538.74569	9859.75884
44	12411.33137	8087.07769	10249.20453
45	11668.40119	8962.15323	10315.27721
46	11621.8658	9152.33279	10387.0993
47	8717.00046	12924.63957	10820.82002
48	8166.22927	13846.12342	11006.17634
49	11415.13698	11550.54846	11482.84272
50	9364.77656	13488.43046	11426.60351

B.1.2 Hasil Pengujian untuk *Query* Terjemahan Bahasa Jawa-Sunda

SINGLE SERVER

Jml User	Jawa	Sunda	Total
1	651.01128	647.85626	649.43377
2	625.71692	627.48869	626.60281
3	626.36223	627.30377	626.833
4	630.50884	628.61868	629.56376
5	712.31636	730.25226	721.28431
6	805.52402	822.10872	813.81637
7	929.25734	956.03202	942.64468
8	1072.86854	1067.34228	1070.10541
9	1175.23043	1182.54994	1178.89019
10	1289.26744	1301.53635	1295.4019
11	1375.39114	1451.21107	1413.30111
12	1498.83392	1499.63271	1499.23332
13	1577.46689	1622.98293	1600.22491
14	1722.00262	1721.03393	1721.51827
15	1826.33983	1815.22003	1820.77993
16	1890.24703	1916.07889	1903.16296
17	1995.46803	1999.37099	1997.41951
18	2091.58669	2099.90282	2095.74476
19	2235.2013	2155.13758	2195.16944
20	2277.64462	2284.62489	2281.13475
21	2401.87568	2400.52005	2401.19787
22	2494.49381	2520.50186	2507.49784
23	2613.52889	2552.73913	2583.13401
24	2721.48817	2707.36845	2714.42831
25	2809.75857	2792.94174	2801.35015
26	2878.88752	2902.38613	2890.63682
27	3023.3681	2979.8453	3001.6067

DISTRIBUTED SERVER

Jml User	Jawa	Sunda	Total
1	667.17072	705.89313	686.53193
2	650.83693	689.31606	670.07649
3	644.18763	674.56286	659.37524
4	643.39661	670.72363	657.06012
5	635.17197	663.98387	649.57792
6	650.92941	667.09371	659.01156
7	672.04488	704.29251	688.1687
8	729.47834	766.19586	747.8371
9	743.81151	790.76858	767.29005
10	834.44078	884.58743	859.5141
11	877.77683	920.08652	898.93167
12	927.39458	985.74546	956.57002
13	987.77649	1082.34022	1035.05835
14	1053.04996	1050.51077	1051.78036
15	1108.17483	1135.40299	1121.78891
16	1157.53542	1146.36104	1151.94823
17	1243.40179	1246.56673	1244.98426
18	1236.26098	1330.11074	1283.18586
19	1267.61279	1332.70822	1300.16051
20	1313.77765	1439.65948	1376.71857
21	1272.63014	1453.83129	1363.23072
22	1435.4962	1474.20496	1454.85058
23	1454.52833	1632.35693	1543.44263
24	1467.92273	1868.63444	1668.27858
25	1689.97118	1534.22421	1612.09769
26	1758.55344	1617.97861	1688.26602
27	1802.5671	1741.0356	1771.8014

	8	9	9
28	3100.3991	3101.80146	3101.10028
29	3241.16048	3176.34102	3208.75075
30	3232.78666	3322.06248	3277.42457
31	3384.83048	3337.99068	3361.41058
32	3520.21266	3509.99451	3515.10358
33	3641.6555	3586.2058	3613.93065
34	3734.6045	3695.43083	3715.01766
35	3814.1526	3806.63478	3810.39369
36	3885.25935	3919.56133	3902.41034
37	4025.10032	3973.29735	3999.19883
38	4127.27229	4141.2777	4134.27499
39	4338.61946	4400.30974	4369.4646
40	5036.93207	5063.65022	5050.29115
41	5311.43247	5406.18669	5358.80958
42	5850.37526	5674.65015	5762.51271
43	5991.57794	6051.75114	6021.66454
44	6362.75558	5987.32986	6175.04272
45	6334.70845	6248.14871	6291.42858
46	6364.69609	6541.75828	6453.22718
47	6526.95476	6427.4444	6477.19958
48	6664.28097	6519.2431	6591.76204
49	6909.54158	7023.30773	6966.42466
50	7063.32983	7008.45252	7035.89118

	2	8	
28	1768.31462	1834.17794	1801.24628
29	1811.62228	1892.01929	1851.82078
30	1982.8316	1763.92717	1873.37938
31	1851.24908	2149.17722	2000.21315
32	1942.63222	2151.5211	2047.07666
33	2114.87493	2006.31734	2060.59614
34	1973.17386	2373.69802	2173.43594
35	2085.17659	2412.03015	2248.60337
36	2075.30794	2439.48244	2257.39519
37	2174.68558	2306.4007	2240.54314
38	2234.52073	2389.19153	2311.85613
39	2335.84345	2452.70289	2394.27317
40	2533.67612	2395.55989	2464.618
41	2122.877	2822.02872	2472.45286
42	2494.9395	2780.39056	2637.66503
43	2686.46312	2559.83388	2623.1485
44	2509.8482	2711.30389	2610.57605
45	2692.02008	2755.29651	2723.65829
46	2402.59972	3074.15019	2738.37496
47	2618.66335	3086.6598	2852.66157
48	2834.56164	2923.84649	2879.20407
49	3041.39661	2882.31204	2961.85432
50	2806.41209	3243.78985	3025.10097